

Taproot + Ordinal Commitment Specification

Universal Bridge — Bitcoin Anchoring Layer

Document ID	UB-SPEC-BTC-001
Version	1.0.0-draft
Status	AUTHORITATIVE DRAFT
Date	09 July 2026
Authors	Universal Bridge Protocol Working Group
Scope	Universal Bridge — Bitcoin Anchoring Layer
References	BIP-340 (Schnorr Signatures), BIP-341 (Taproot), BIP-342 (Tapscript), Ordinal Theory (Casey Rodarmor)

Abstract

This specification defines the cryptographic primitives, data structures, wire formats, and protocol rules governing how the Universal Bridge commits cross-chain state to Bitcoin using Taproot outputs and Ordinal Satoshi bindings. It is the single canonical reference for implementers of the Bitcoin anchoring layer.

The Universal Bridge employs a two-phase commit-reveal scheme: a Taproot output whose output key encodes a cryptographically bound commitment hash is published on-chain in Phase 1 (Commit), and the full payload together with a structured witness is revealed in Phase 2 (Reveal). An Ordinal Satoshi binding associates a specific globally-unique Satoshi — identified by its ordinal number — with the commitment payload, providing provenance, ordering, and anti-replay guarantees without requiring any modifications to the Bitcoin consensus rules.

Taproot + Ordinal Commitment Specification

Universal Bridge (URIB)

Deterministic Commitment, Satoshi-Level Anchoring, and Cross-Rail Truth Binding

NORMATIVE REQUIREMENT

Developers

MUST

implement every section of this specification to claim compliance with the Universal Bridge Bitcoin Anchoring Layer. Partial implementations are not considered conformant. Deviations

MUST

be documented in an errata filed with the Protocol Working Group before deployment.

Table of Contents

1. Foundations and Terminology Notation Glossary
2. Taproot Tweak Mathematics Internal Key Selection TapLeaf Hash Construction TapBranch Hash Construction TapTree Merkle Root Computation The Taproot Tweak Universal Bridge Tweak Extension Key-Path vs. Script-Path Spending
3. Ordinal Assignment Rules Ordinal Theory Primer FIFO Transfer Rule Ordinal Rarity Classes Universal Bridge Sat Selection Policy Ordinal Stability Guarantee
4. Satoshi Binding Model The UBCommitRecord Structure Flags Bitfield Sat-to-Payload Binding Binding Verification Algorithm Dust and Value Rules
5. Commit / Reveal Pattern Overview Commit Transaction Structure Reveal Transaction Structure UB Commitment Script Template Annex Encoding Fee Estimation RBF and Replace-by-Fee Policy
6. Cross-Rail Anchoring Behavior What Is Cross-Rail Anchoring? Multi-Rail Payload Structure Anchor Finality Rules Anchor Chain of Custody Anchor Reorg Handling Cross-Rail Finality State Machine Emergency Recovery Script
7. Implementation Reference Canonical Parameter Table Tagged Hash Domains Test Vectors Security Considerations Compliance Checklist
8. Appendix A — BIP-341 Reference Pseudocode
9. Appendix B — Chain ID Registry
10. Appendix C — Glossary Index

Taproot + Ordinal Commitment Specification

Universal Bridge (URIB)

Deterministic Commitment, Satoshi-Level Anchoring, and Cross-Rail Truth Binding

Preamble

The Universal Bridge Bitcoin Anchoring Layer provides a trust-minimized mechanism for committing cross-chain state to Bitcoin by exploiting two independent Bitcoin protocol features: Taproot (BIP-341), which allows arbitrary data to be committed inside a Taproot output key without revealing it until spend-time; and Ordinal Theory, which assigns globally unique identifiers to individual satoshis and tracks their transfer via a deterministic FIFO algorithm.

Together, these primitives yield a commitment scheme with the following properties:

- **Binding:** The committed payload is cryptographically bound to a specific on-chain output. No alternative payload can satisfy the same output key without knowledge of the internal key and the original payload.
- **Hiding:** Until the reveal transaction is broadcast, observers see only a standard P2TR output indistinguishable from a key-path spend.
- **Ordinal Provenance:** By associating the commitment with a specific Satoshi, the protocol inherits ordinal-level ordering, uniqueness, and transfer traceability.
- **Script Enforceability:** When the script-path variant is used, the commitment is additionally enforced by Tapscript, enabling timelocks, multi-sig, and dispute resolution entirely within Bitcoin's consensus rules.

This document is organized into seven normative sections followed by three informational appendices. Appendices do not impose additional requirements but provide reference material that implementers should consult.

The key words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are to be interpreted as described in RFC 2119.

1. Foundations and Terminology

1.1 Notation

All mathematical notation used throughout this specification is defined below. Implementers **MUST** interpret every symbol exactly as defined here; no alternative interpretation is permissible.

Symbol / Function	Definition
p	The secp256k1 field prime: $p = 0xFFFEFFFFC2F$
n	The secp256k1 group order: $n = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141$
G	The secp256k1 generator point (standard, as defined in SEC 2)
$k \cdot G$	Scalar multiplication of generator G by integer k
$\text{tagged_hash}(tag, msg)$	$\text{SHA256}(\text{SHA256}(tag) \text{SHA256}(tag) msg)$ — domain-separated hash per BIP-340
$\text{bytes}(x)$	32-byte big-endian encoding of scalar integer x
$\text{bytes8_BE}(x)$	8-byte big-endian encoding of integer x
$\text{bytes4_BE}(x)$	4-byte big-endian encoding of integer x
$\text{int}(b)$	Integer decoded from 32-byte big-endian buffer b
$\text{lift_x}(x)$	The unique point P on secp256k1 with x-coordinate x and even Y-coordinate; undefined if no such point exists
$\text{has_even_y}(P)$	true iff $Y(P) \bmod 2 == 0$
$\text{bytes_x}(P)$	The 32-byte x-coordinate of point P
$X(P), Y(P)$	The x- and y-coordinates of elliptic curve point P respectively
$\oplus$	Bitwise XOR of byte strings of equal length
$ $	Byte concatenation

Taproot + Ordinal Commitment Specification

Universal Bridge (URIB)

Deterministic Commitment, Satoshi-Level Anchoring, and Cross-Rail Truth Binding

Symbol / Function	Definition
<code>compact_size(n)</code>	Bitcoin variable-length integer encoding of <code>n</code> (1, 3, 5, or 9 bytes)
<code>H_TapTweak</code>	<code>tagged_hash</code> with tag string "TapTweak" (BIP-341)
<code>H_TapLeaf</code>	<code>tagged_hash</code> with tag string "TapLeaf" (BIP-341)
<code>H_TapBranch</code>	<code>tagged_hash</code> with tag string "TapBranch" (BIP-341)
<code>H_UBCommit</code>	<code>tagged_hash</code> with tag string "UB/Commit/v1" — Universal Bridge extension; not defined in any BIP
<code>H_UBOrdBind</code>	<code>tagged_hash</code> with tag string "UB/OrdBind/v1" — Universal Bridge extension

NOTE — Tagged Hash Security Domain

The double-SHA256 tag prefix in

`tagged_hash`

ensures that hash outputs in different protocol contexts (e.g.,

`H_TapTweak`

vs.

`H_UBCommit`

) are drawn from distinct hash domains, preventing cross-protocol collision attacks.

Implementers

MUST NOT

reuse any existing BIP tag string for a new hash domain.

Taproot + Ordinal Commitment Specification

Universal Bridge (URIB)

Deterministic Commitment, Satoshi-Level Anchoring, and Cross-Rail Truth Binding

1.2 Glossary

All terms below carry the exact meaning defined here throughout this specification.

Term	Definition
Internal Key (P)	The 32-byte x-only secp256k1 public key that is tweaked to produce the output key. P is never directly committed to the scriptPubKey.
Output Key (Q)	The tweaked x-only secp256k1 public key embedded in the P2TR scriptPubKey; $Q = P + t \cdot G$ normalized to even Y.
TapLeaf	A leaf node in the Tapscript Merkle tree; contains a one-byte leaf version and a variable-length script, hashed via $H_TapLeaf$.
TapBranch	An interior node of the TapTree; the hash of two child nodes concatenated in lexicographic order, hashed via $H_TapBranch$.
TapTree (T)	The full binary Merkle tree of Tapscript spending conditions from which the Merkle root h is derived.
Merkle Root (h)	The 32-byte root hash of the TapTree T; used as input to the Taproot tweak equation.
Commitment Payload (C)	The Universal Bridge cross-chain state blob being anchored to Bitcoin; can be any length and is hashed before being committed.
Ordinal (ord)	A unique 64-bit unsigned integer assigned to each satoshi in the exact order it was mined, beginning at 0 for the first sat of the genesis coinbase.
Satoshi Binding	The cryptographic association of a specific ordinal satoshi — identified by its ordinal number — to a commitment payload via $H_UBOrdBind$, enforced through the Taproot output key.
Reveal Transaction	The Bitcoin transaction that spends the commit UTXO and exposes the full UBCommitRecord, payload, and (optionally) the Tapscript in its witness data.
Anchor UTXO	The Bitcoin transaction output (P2TR) created in Phase 1 whose scriptPubKey encodes the Universal Bridge commitment hash.
Cross-Rail Anchor	An anchor UTXO whose commitment payload simultaneously references finalized state on two or more non-Bitcoin blockchain rails.

Taproot + Ordinal Commitment Specification

Universal Bridge (URIB)

Deterministic Commitment, Satoshi-Level Anchoring, and Cross-Rail Truth Binding

Term	Definition
NUMS Point	A Nothing-Up-My-Sleeve point: a secp256k1 public key with no known discrete logarithm, used as the internal key when key-path spending is intentionally disabled.
Control Block	The witness stack element required for a script-path spend; contains the leaf version, the parity bit of Q, the internal key P, and the Merkle proof path.
Parity Bit	A single bit (0 or 1) indicating whether Q had even (0) or odd (1) Y before normalization; encoded in the first byte of the control block.
Dust Limit	The minimum output value (in satoshis) below which a Bitcoin node will reject an output as non-standard. For P2TR, the dust limit is 546 satoshis.
Stability Window	The minimum number of Bitcoin block confirmations that must accrue on the commit UTXO before the reveal transaction may be broadcast. Defined as 6 blocks in this specification.
Rail	A distinct blockchain network participating in the Universal Bridge (e.g., Bitcoin, Ethereum, Solana, Cosmos Hub).
bind_hash	The 32-byte output of H_UBOrdBind over the sat ordinal and payload fields; used as h in the Taproot tweak when satoshi binding is active.

2. Taproot Tweak Mathematics

2.1 Internal Key Selection

The internal key P is the secp256k1 public key that, after tweaking, becomes the output key committed to the blockchain. Selection of P has significant security implications and **MUST** follow the rules below.

- P **MUST** be a valid x-only secp256k1 public key represented as exactly 32 bytes (the x-coordinate), with an implied even Y-coordinate.
- P **SHOULD** be derived from a BIP-32 HD derivation path:

where `anchor_index` is a monotonically increasing 32-bit counter unique per bridge operator key.

- Mainnet: `m/86'/0'/0'/anchor_index` (BIP-86 Taproot derivation)
- Testnet: `m/86'/1'/0'/anchor_index`
- P **MAY** be a NUMS point when key-path spending is intentionally disabled. The canonical Universal Bridge NUMS point is defined as:

```
NUMS_CANDIDATE = lift_x(SHA256("UB/NUMS/internal-key/v1"))  If NUMS_CANDIDATE
is a valid curve point:      P_NUMS = NUMS_CANDIDATE Else for i = 1, 2, 3,
...:      NUMS_CANDIDATE = lift_x(SHA256("UB/NUMS/internal-key/v1/" ||
str(i)))      If NUMS_CANDIDATE is valid:      P_NUMS = NUMS_CANDIDATE
break
```

The counter suffix approach guarantees a deterministic, reproducible NUMS point that every implementation will agree on. This point has no known discrete logarithm because its derivation is publicly auditable hash output — if a discrete logarithm existed and were known, the SHA256 preimage resistance would be broken.

Taproot + Ordinal Commitment Specification

Universal Bridge (URIB)

Deterministic Commitment, Satoshi-Level Anchoring, and Cross-Rail Truth Binding

- When a key-path spend *is* intended (standard bridge co-sign flow), P **MUST** be the bridge operator's aggregated MuSig2 public key, derived and verified per the MuSig2 specification (BIP-327).
- P **MUST NOT** be the point at infinity.
- P **MUST NOT** be reused across distinct commitment nonces without explicit key rotation.

2.2 TapLeaf Hash Construction

Each Tapscript leaf in the TapTree is serialized and hashed as follows. All Universal Bridge tap-leaves **MUST** use leaf version `0xC0`.

```
tapleaf_hash(version, script):      # version: 1 byte, MUST be 0xC0 for UB
leaves      # script: variable-length byte array (the locking script)
return H_TapLeaf(      bytes([version])      ||      # 1 byte leaf version
compact_size(len(script)) ||      # 1-9 bytes (almost always 1 byte for UB
scripts)      script      # N bytes      )      # Result:
32 bytes      Constraint:      version MUST be 0xC0 (decimal 192)      len(script)
MUST be > 0 and <= data-id="400" 10000 bytes      compact_size encoding MUST
match Bitcoin standard (not padded)
```

Taproot + Ordinal Commitment Specification

Universal Bridge (URIB)

Deterministic Commitment, Satoshi-Level Anchoring, and Cross-Rail Truth Binding

CRITICAL — Leaf Version

Using any leaf version other than

0xC0

for Universal Bridge tap-leaves is non-compliant and will result in control blocks that fail consensus validation on all currently deployed Bitcoin nodes. Future leaf versions are reserved for protocol upgrades and

MUST NOT

be used without a new version of this specification.

2.3 TapBranch Hash Construction

Interior nodes of the TapTree are formed by combining two child hashes. The lexicographic ordering ensures the tree is canonical regardless of the order in which leaves were inserted.

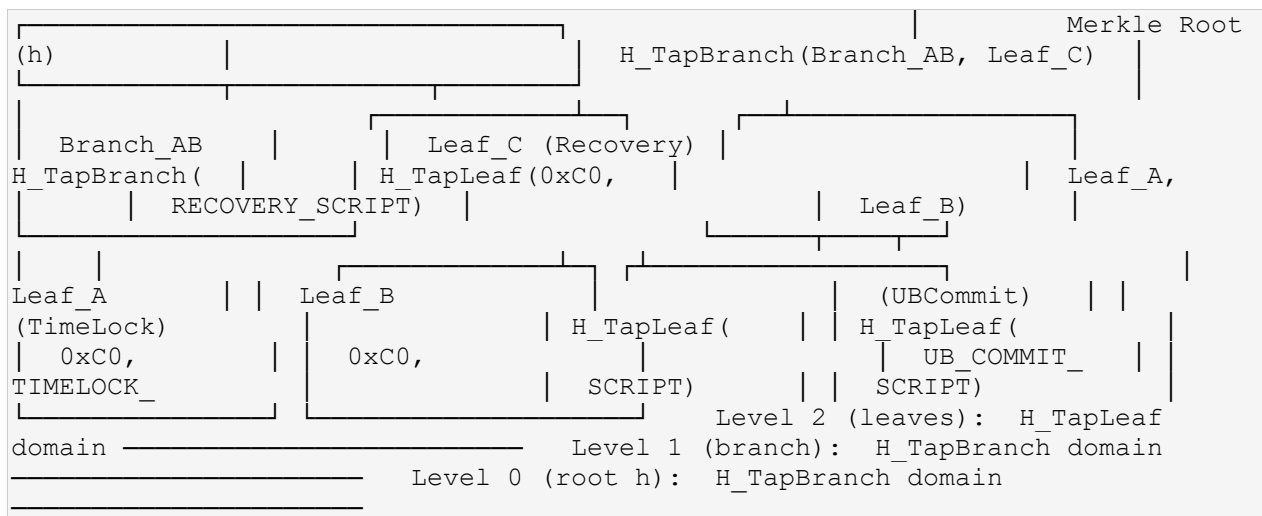
```
tapbranch_hash(left, right):      # left, right: each a 32-byte hash (either
tapleaf_hash or tapbranch_hash)  if left <= data-id="408" right:      #
lexicographic byte comparison    return H_TapBranch(left || right)
else:                             return H_TapBranch(right || left)    # Result: 32 bytes
Property:      tapbranch_hash(a, b) == tapbranch_hash(b, a)    # commutative by
design         This ensures the Merkle root is unique for a given set of leaves
regardless of tree construction order.
```

2.4 TapTree Merkle Root Computation

Given a set of tap-leaves, the Merkle root h is computed by iteratively combining leaves and branches until a single 32-byte root remains. The algorithm below is the canonical procedure for Universal Bridge TapTree construction.

```
TapTreeMerkleRoot(leaves):      # leaves: list of (depth, script_version,
script) triples                  # Step 1: Compute all tapleaf_hash values
nodes = [(depth, tapleaf_hash(ver, script)) for (depth, ver, script) in leaves]
# Step 2: Sort by depth descending (deepest leaves first)
nodes.sort(key=lambda x: -x[0])  # Step 3: Combine sibling pairs
iteratively while len(nodes) > 1: # Combine the two deepest nodes
(must be siblings at same depth) right_depth, right_hash =
nodes.pop(0) left_depth, left_hash = nodes.pop(0) assert
left_depth == right_depth, "Sibling depth mismatch" parent =
(left_depth - 1, tapbranch_hash(left_hash, right_hash))
nodes.append(parent) assert len(nodes) == 1, "Reduction failed"
_, h = nodes[0] return h # 32-byte Merkle root
```

The diagram below illustrates the three-leaf TapTree used in the standard Universal Bridge commitment, with hash domain labels at each level:



Taproot + Ordinal Commitment Specification

Universal Bridge (URIB)

Deterministic Commitment, Satoshi-Level Anchoring, and Cross-Rail Truth Binding

2.5 The Taproot Tweak

The Taproot tweak is the core mathematical operation that embeds the Merkle root (or Universal Bridge commitment hash) into the output key. Implementers **MUST** follow this procedure exactly, including the even-Y normalization step.

```
TaprootOutputKey(P, h):      # P: 32-byte x-only internal key (implied even Y)
# h: 32-byte Merkle root (or H_UBCommit output in keypath-only mode)      #
Step 1: Compute tweak scalar  t = int(H_TapTweak(bytes(P) || h))      # 32-
byte tagged hash → integer      # Step 2: Validate tweak      MUST verify: t <
n      # If t >= n, the script tree is invalid. Change a leaf script or nonce
# and retry. This failure is astronomically rare (probability ~2^-128).
# Step 3: Compute tweaked point  Q = lift_x(int(bytes(P))) + t · G      #
Note: lift_x ensures P has even Y before point addition      # Step 4: Even-Y
normalization      if has_even_y(Q):      output_key = bytes_x(Q)      # 32
bytes, x-coordinate of Q      parity_bit = 0      else:      output_key
= bytes_x(-Q)      # negate Q; -Q has same x-coord, flipped Y
parity_bit = 1      # NOTE: The output key is always the x-coordinate;
negation only      # affects the parity_bit recorded in the control block
for script-path spends      return (output_key, parity_bit)
```

The final scriptPubKey committed to the Bitcoin blockchain is a Pay-to-Taproot (P2TR) output:

```
scriptPubKey = OP_1 <output_key>      = 0x51 || 0x20 || output_key
# 0x51 = OP_1 (push version 1)      # 0x20 = PUSH_32 (push 32 bytes)
# output_key = 32-byte x-only tweaked public key      # Total: 34
bytes
```

NOTE — Implied Even-Y Convention

BIP-341 mandates that the output key embedded in scriptPubKey always corresponds to the even-Y representative of the secp256k1 point. The parity bit is NOT included in the scriptPubKey; it is only relevant for constructing the control block during a script-path spend. Implementers building key-path spend signers

MUST

negate the private key if

Taproot + Ordinal Commitment Specification

Universal Bridge (URIB)

Deterministic Commitment, Satoshi-Level Anchoring, and Cross-Rail Truth Binding

```
parity_bit == 1
```

, so that the signing key matches the embedded output key.

2.6 Universal Bridge Tweak Extension

When committing a Universal Bridge payload C in keypath-only mode (no script tree), the TapTree is empty and the commitment is embedded directly via the `H_UBCommit` tagged hash. This mode is selected when the `KEY_PATH_ONLY` flag (bit 1) is set in `UBCommitRecord.flags`.

```
UBKeyPathCommit(P, C):      # P: 32-byte x-only internal key (bridge
operator's MuSig2 key)      # C: variable-length commitment payload bytes
# Step 1: Hash the payload with UB domain tag      h_ub = H_UBCommit(C)
# 32-byte commitment hash      # Step 2: Use h_ub as if it were the TapTree
Merkle root      t = int(H_TapTweak(bytes(P) || h_ub))      MUST verify: t < n
# Step 3: Compute and normalize output key (identical to Section 2.5)      Q =
lift_x(int(bytes(P))) + t · G      if has_even_y(Q):      output_key =
bytes_x(Q)      parity_bit = 0      else:      output_key = bytes_x(-Q)
parity_bit = 1      return (output_key, parity_bit, h_ub)      # h_ub is
stored in reveal witness annex for later verification      Verification:      #
Anyone who knows P and C can recompute h_ub and verify output_key matches.
# The payload C is revealed in the reveal transaction's witness annex (see
$5.5).      # No control block is needed; the reveal is a key-path spend.
```

2.7 Key-Path vs. Script-Path Spending

The Universal Bridge supports both spending modes. The appropriate mode is selected based on the bridge operation type. Implementers **MUST** support both modes.

Property	Key-Path Spend	Script-Path Spend
Witness structure	64-byte Schnorr signature only	Signature + script + control block [+ annex]
Approximate witness size	~65 vBytes	Variable; typically 150–400+ vBytes depending on script and Merkle path depth
Privacy	High — indistinguishable from any P2TR key-path spend on-chain	Lower — reveals leaf script, control block, and Merkle path
Universal Bridge use case	Standard bridge co-sign; happy-path reveals; fungible token bridges	Dispute resolution; timelocked exits; emergency recovery; NFT provenance bridges
Requires TapTree	No (h may be H_UBCommit(C) directly)	Yes — at minimum a UBCommit leaf and a Recovery leaf
Control block present	Absent	Present: (leaf_version parity_bit) internal_key(32) merkle_path(0–128 bytes)
Internal key disclosure	Not revealed (P remains private)	Internal key P is revealed in control block
Fee cost	Minimum (~111 vBytes reveal tx)	Higher; proportional to annex + script + path length

3. Ordinal Assignment Rules

3.1 Ordinal Theory Primer

Ordinal Theory, as defined by Casey Rodarmor's *ord* protocol, assigns a globally unique 64-bit unsigned integer (the "ordinal") to every satoshi in the order it was mined into existence. The assignment is deterministic, requires no on-chain state beyond the Bitcoin blockchain itself, and is never invalidated by subsequent transactions (ordinals transfer with their satoshi, they do not expire).

Key definitions:

- Ordinal numbering begins at **0**, assigned to the first satoshi of the genesis block coinbase output.
- Ordinals are assigned sequentially within a block: coinbase outputs are assigned before any transaction outputs in that block.
- The subsidy schedule governing how many new ordinals are created per block is:

```
block_subsidy(height):      # Returns the block subsidy in satoshis at the
given block height        halvings = floor(height / 210000)      return
floor(50000000000 / 2^halvings)  # height=0:      5,000,000,000 sat (50
BTC)      # height=210000:  2,500,000,000 sat (25 BTC)      # height=420000:
1,250,000,000 sat (12.5 BTC)      # height=840000:    625,000,000 sat (6.25
BTC)      # height=1050000:  312,500,000 sat (3.125 BTC)      # ... continues
halving every 210,000 blocks
first_sat(block_height b):    # Returns the ordinal of the first sat created
in block b      # Ignoring fee sats (which inherit ordinals via FIFO from
inputs)      return sum_{i=0}^{b-1} block_subsidy(i)      # Example:
first_sat(0) = 0      # Example: first_sat(1) = 5,000,000,000      # Example:
first_sat(210000) = 1,050,000,000,000,000
```

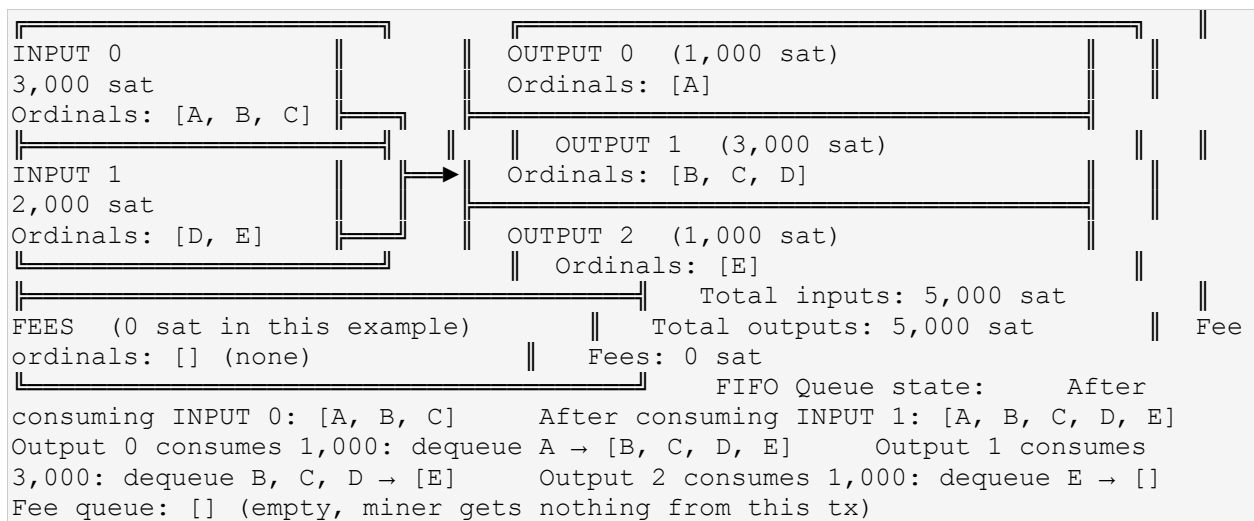
Fee satoshis do not receive new ordinal numbers. Instead, they carry the ordinals of the input satoshis that were consumed but not assigned to any output, per the FIFO rule defined in Section 3.2. The coinbase transaction of each block may then collect those ordinals as part of the miner's fee income.

3.2 FIFO Transfer Rule

Ordinals flow through Bitcoin transactions via a strict First-In, First-Out (FIFO) rule applied across all inputs and outputs in index order. This rule is deterministic and stateless given the full blockchain history.

```
OrdinalFlow(tx):      # Step 1: Build ordered queue of all input ordinals
input_queue = deque()  for input in tx.inputs (in index order 0, 1, 2,
...):                  spent_utxo = resolve_utxo(input.txid, input.vout)
input_ordinals = ordinals_of_utxo(spent_utxo)  # sorted ascending
for ord in input_ordinals:      input_queue.append(ord)      # Step 2:
Assign ordinals to outputs in index order      output_assignments = []  for
output in tx.outputs (in index order 0, 1, 2, ...):          assigned = []
for _ in range(output.value):      # value in satoshis          if
input_queue:                  assigned.append(input_queue.popleft())
# If input_queue exhausted: this output receives newly minted sats
# (only possible for coinbase transactions)
output_assignments.append((output.index, assigned))      # Step 3: Remaining
ordinals → miner fees      fee_ordinals = list(input_queue)      # These appear
in the coinbase tx of the same block via the same FIFO rule      return
output_assignments, fee_ordinals
```

The following diagram illustrates a concrete example of ordinal flow through a transaction with two inputs and three outputs:



3.3 Ordinal Rarity Classes

Ordinal Theory defines a set of rarity classes based on the significance of the block or epoch in which a satoshi was first mined. The Universal Bridge uses these classes for sat selection policy (Section 3.4).

Rarity Token	Human Name	Defining Rule	Approximate Count
<code>common</code>	Common	Any satoshi not meeting any criteria below. Ordinal is not the first sat of any block, difficulty epoch, halving epoch, or cycle.	~2.1 quadrillion (virtually all sats)
<code>uncommon</code>	Uncommon	The first satoshi of each block; ordinal == <code>first_sat(b)</code> for some block <code>b</code> .	~1 per block; ~900,000+ at current height
<code>rare</code>	Rare	The first satoshi of each difficulty adjustment period, occurring every 2,016 blocks.	~436 at current height; 3,437 total cap
<code>epic</code>	Epic	The first satoshi of each halving epoch; ordinal == <code>first_sat(b)</code> where <code>b</code> is a halving block (0, 210000, 420000, ...)	33 total (across all halvings until supply exhausted)
<code>legendary</code>	Legendary	The first satoshi of each cycle, defined as every 6 halving epochs (every 1,260,000 blocks).	5 total
<code>mythic</code>	Mythic	The very first satoshi ever mined: ordinal 0, in the genesis block coinbase output.	1 — unique, unspendable by Satoshi's key

3.4 Universal Bridge Sat Selection Policy

The following rules govern which satoshi is selected as the ordinal carrier for a Universal Bridge commitment. These rules are evaluated in precedence order; the first applicable rule applies.

1. If the bridge operation involves an NFT or non-fungible asset crossing rails (i.e., provenance matters), the selected sat **MUST** have rarity $\geq$ `uncommon`. The sat's rarity **SHOULD** be at least as rare as the rarity class of the asset being bridged (e.g., an epic-inscribed NFT **SHOULD** use an epic or rarer sat).
2. For fungible token bridges and generic state commitments with no provenance requirement, any `common` sat **MAY** be used as the carrier.
3. The selected carrier sat **MUST** occupy output index 0 of the commit transaction (the inscription output). Any sat that would be assigned to an output other than index 0 by the FIFO rule **MUST NOT** be selected as the carrier without restructuring the commit transaction inputs.
4. The carrier sat's ordinal number **MUST** be recorded in `UBCommitRecord.sat_ordinal` before the record is signed (see Section 4.1).
5. If the required rarity sat is unavailable in the operator's wallet at the time of commitment, the operator **MUST NOT** substitute a lower-rarity sat. The commitment transaction **MUST** be deferred until a sat of the required rarity is available or acquired.
6. The operator **SHOULD** maintain a reserve of uncommon sats (at least one per expected concurrent commitment) to avoid availability bottlenecks.

3.5 Ordinal Stability Guarantee

The Universal Bridge protocol requires that the carrier satoshi remain stable — that is, not at risk of being inadvertently split, merged, or swept — from the moment of the commit transaction through the reveal.

- The commit UTXO **MUST** be held for at least **6 blocks** (the stability window) before the reveal transaction is broadcast. This prevents ordinal displacement due to shallow reorgs.
- For cross-rail state that is considered irreversible on the source rail (e.g., a finalized ETH withdrawal), Bitcoin confirmation requirements before finalizing on the destination rail are:

Scenario	Required Bitcoin Confirmations	Rationale
Standard fungible bridge (BTC destination)	6	Industry-standard probabilistic finality
High-value transfer (>10 BTC equivalent)	24	Increased reorg depth protection
Irreversible source-rail event (ETH→BTC)	100	Exceeds historical maximum Bitcoin reorg depth with large margin
NFT / non-fungible asset bridge	6 (reveal) + 100 (finalize)	Ordinal provenance requires maximum reorg safety

- The operator's node **MUST** monitor for chain reorgs using a rolling window of 100 blocks. See Section 6.5 for reorg handling procedures.
- The commit UTXO **MUST NOT** be used as an input to any other transaction while awaiting reveal. Doing so would reassign the carrier sat's ordinal via the FIFO rule and invalidate the binding.

4. Satoshi Binding Model

4.1 The UBCCommitRecord Structure

The `UBCommitRecord` is the canonical fixed-length serialization of all Universal Bridge commitment metadata. It is a binary structure with no variable-length fields; all multi-byte integers use big-endian byte order unless explicitly stated otherwise.

UBCommitRecord (167 bytes, fixed-length):			
Field	Offset	Description	
magic	0- 3	4 bytes = 0x55 0x42 0x43 0x00 ("UBC\x00")	
version	4	1 byte = 0x01 (this spec version)	
flags	5	1 byte bitfield; see Section 4.2	
chain_id_src	6- 9	4 bytes source chain ID, big-endian	
chain_id_dst	10-13	4 bytes destination chain ID, big-endian	
nonce	14-21	8 bytes unique per commitment, big-endian	
payload_hash	22-53	32 bytes SHA256(full_state_payload_bytes)	
sat_ordinal	54-61	8 bytes ordinal of carrier sat, big-endian	
operator_pubkey	62-94	33 bytes compressed secp256k1 pubkey of operator	
timestamp	95-98	4 bytes Unix timestamp of commit (big-endian)	
expiry	99-102	4 bytes Unix timestamp of commitment expiry (BE)	
signature	103-166	64 bytes BIP-340 Schnorr signature over SHA256(record[0:103])	
Total: 167 bytes Constraints:			
magic		MUST equal 0x55424300 exactly	
version		MUST equal 0x01 for this specification	
flags bits 5-7		MUST be zero	
sat_ordinal		MUST NOT be zero (ordinal 0 is the mythic genesis sat; its use requires explicit rarity=mythic flag handling)	
expiry		MUST be > timestamp (expiry in the future at commit time)	
signature		MUST be a valid BIP-340 Schnorr signature by the key in SHA256(record[0:103])	
operator_pubkey		over	

Taproot + Ordinal Commitment Specification

Universal Bridge (URIB)

Deterministic Commitment, Satoshi-Level Anchoring, and Cross-Rail Truth Binding

ALIGNMENT NOTE

The

UBCommitRecord

is intentionally fixed at 167 bytes with no padding, alignment bytes, or length prefix.

Parsers

MUST NOT

assume alignment of any field. All multi-byte reads

MUST

use explicit big-endian byte extraction regardless of host architecture endianness.

Taproot + Ordinal Commitment Specification

Universal Bridge (URIB)

Deterministic Commitment, Satoshi-Level Anchoring, and Cross-Rail Truth Binding

4.2 Flags Bitfield

The `flags` byte at offset 5 is a bitfield. Each bit controls a specific behavioral modifier for the commitment.

Bit	Mask	Name	Meaning When Set (= 1)
0	0x01	CROSS_RAIL	This commitment spans two or more non-Bitcoin rails. The payload MUST conform to the MultiRailPayload structure (Section 6.2).
1	0x02	KEY_PATH_ONLY	No TapScript tree is present. The commitment is embedded via H_UBCommit in keypath-only mode (Section 2.6). The reveal MUST be a key-path spend.
2	0x04	RARITY_REQUIRED	The carrier sat MUST have rarity $\geq$ <code>uncommon</code> . Validators MUST check this against the FIFO-traced ordinal.
3	0x08	TIMELOCK	The reveal is timelocked. The <code>expiry</code> field is authoritative; the reveal tx MUST set <code>nLockTime</code> accordingly and the UB_COMMIT_SCRIPT MUST include OP_CHECKLOCKTIMEVERIFY.
4	0x10	MULTI_SIG	The <code>operator_pubkey</code> is a MuSig2 aggregate public key (BIP-327). The signing procedure MUST use the MuSig2 signing protocol, not single-key Schnorr.
5	0x20	RESERVED	MUST be zero. Non-zero values indicate a future protocol version.
6	0x40	RESERVED	MUST be zero.
7	0x80	RESERVED	MUST be zero.

4.3 Sat-to-Payload Binding

The sat-to-payload binding creates a cryptographic link between a specific ordinal satoshi and the commitment payload. This binding is embedded as the Taproot tweak's Merkle root h , making the output key Q uniquely determined by both the carrier sat's ordinal and the payload content. Retroactive commitment fraud is computationally infeasible because altering either the ordinal or the payload changes h , which changes Q , which no longer matches the on-chain UTXO's scriptPubKey.

```
SatPayloadBind(sat_ordinal, chain_id_src, chain_id_dst, payload_hash,
timestamp):    bind_hash = H_UBOrdBind(          bytes8_BE(sat_ordinal)    ||
# 8 bytes: carrier sat ordinal          bytes4_BE(chain_id_src)    ||    # 4
bytes: source chain ID          bytes4_BE(chain_id_dst)    ||    # 4 bytes:
destination chain ID          payload_hash                ||    # 32 bytes: SHA256
of full payload          bytes4_BE(timestamp)              # 4 bytes: Unix commit
timestamp                )    # Total input to H_UBOrdBind: 52 bytes    # Output: 32-
byte bind_hash            return bind_hash                # bind_hash is used as h in
TaprootOutputKey(P, h) [Section 2.5]    # This ensures Q encodes BOTH the
sat identity AND the payload content.
```

Properties of this construction:

- Changing the `sat_ordinal` by even 1 produces a completely different `bind_hash` and thus a different Q . An adversary cannot substitute a different sat without producing a distinct UTXO.
- Changing any byte of `payload_hash` produces a different `bind_hash`. Payload substitution is detectable by recomputing `bind_hash` and comparing against `bytes_x(Q)`.
- The `timestamp` field prevents two otherwise identical commitments (same sat, same payload, same chains) from producing the same `bind_hash`, providing temporal uniqueness.
- The domain tag "UB/OrdBind/v1" in `H_UBOrdBind` ensures outputs are segregated from all BIP-341 tagged hashes and all other UB tagged hashes.

4.4 Binding Verification Algorithm

The following pseudocode defines the normative verification procedure for a sat-payload binding. All steps **MUST** be performed in order; any failure at any step causes the binding to be rejected.

```

verify_binding(utxo, record, payload):      # Inputs:      #   utxo:    the on-
chain UTXO being verified (contains script and value)      #   record:  parsed
UBCommitRecord (167 bytes)      #   payload: the raw payload bytes whose
SHA256 matches record.payload_hash      #   P:        the known internal key
for this commitment session      # Step 1: Confirm output is P2TR      ASSERT
utxo.scriptPubKey[0:2] == bytes([0x51, 0x20])      # OP_1 PUSH_32      # Step
2: Extract output key from scriptPubKey      output_key =
utxo.scriptPubKey[2:34]      # 32 bytes      # Step 3: Validate record magic and
version      ASSERT record.magic == 0x55424300      ASSERT record.version ==
0x01      ASSERT record.flags & 0xE0 == 0x00      # reserved bits must be zero
# Step 4: Validate payload hash      ASSERT SHA256(payload) ==
record.payload_hash      # Step 5: Recompute bind_hash from record fields
bind_hash = H_UBOrdBind(      bytes8_BE(record.sat_ordinal) ||
bytes4_BE(record.chain_id_src) ||      bytes4_BE(record.chain_id_dst) ||
record.payload_hash      ||      bytes4_BE(record.timestamp)      )
# Step 6: Recompute tweak scalar and output key      t =
int(H_TapTweak(bytes(P) || bind_hash))      ASSERT t < n
# tweak validity      Q = lift_x(int(bytes(P))) + t * G      if not
has_even_y(Q):      Q = -Q      computed_key = bytes_x(Q)      # 32
bytes      # Step 7: Assert output key matches      ASSERT computed_key ==
output_key      #
# BINDING VALID: output_key is cryptographically bound to      # (sat_ordinal,
chain_id_src, chain_id_dst, payload_hash)      #
# Step 8:
Verify operator signature      sig_message = SHA256(record[0:103])      # bytes
0-102 inclusive      ASSERT BIP340_verify(record.operator_pubkey, sig_message,
record.signature)      # Step 9: Verify sat ordinal assignment via FIFO trace
commit_tx_ordinals = OrdinalFlow(commit_tx)      output_0_ordinals =
commit_tx_ordinals[0][1]      # ordinals of output index 0      ASSERT
record.sat_ordinal in output_0_ordinals      # Step 10: Verify stability
window      current_height = bitcoin_chain_tip_height()      commit_block =
get_block_height_of_tx(commit_tx.txid)      ASSERT current_height >=
commit_block + 6      # Step 11: Verify record expiry has not passed
ASSERT unix_now() < record.expiry      return VALID

```

4.5 Dust and Value Rules

The value of the commit output (output index 0) is constrained by the P2TR dust limit and the ordinal carrier requirements.

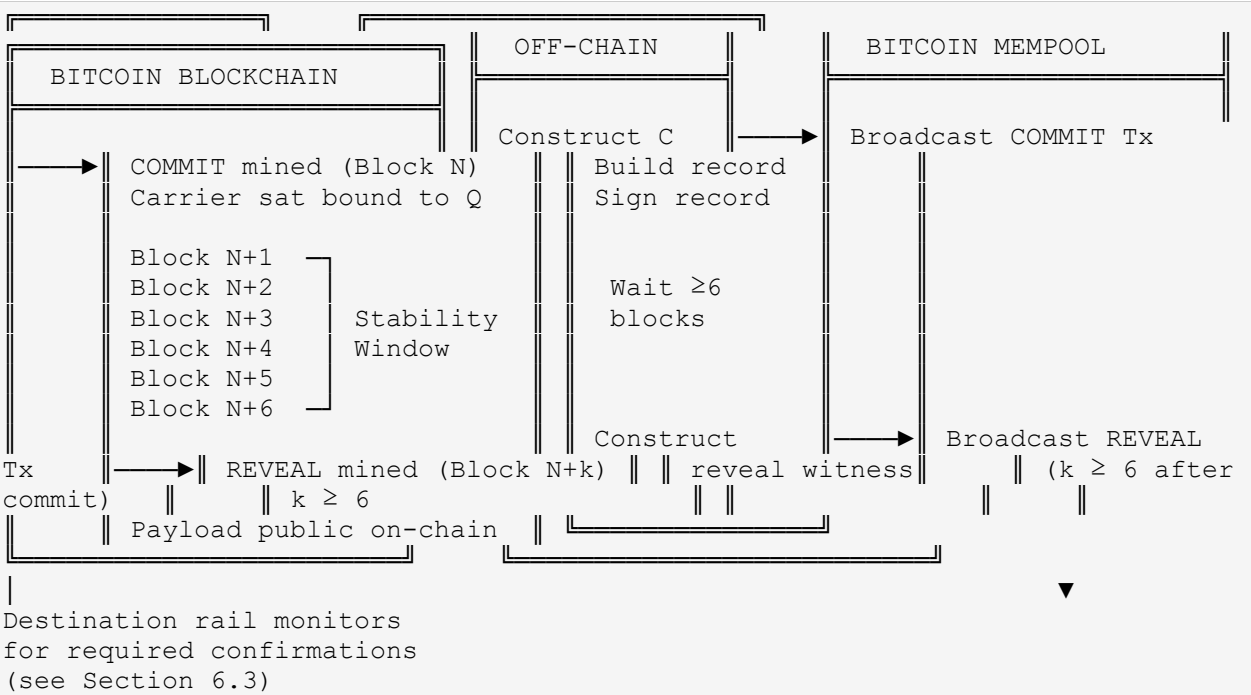
- For `rarity=common` carriers: The commit output **MUST** carry exactly **546 satoshis** — the P2TR dust limit. Among the 546 sats assigned to output 0 by FIFO, the protocol designates the *last* sat (the one with the highest ordinal assigned to that output, i.e., sat index 545 within the output) as the carrier. This is the canonical "546 sat inscription" pattern familiar from Ordinals-based protocols.
- For `rarity ≥ uncommon` carriers: The commit output **MUST** carry exactly **1 Satoshi** (minimal carry). The single sat of that 1-satoshi output is the carrier. The remaining value from the funding input **MUST** be directed to a separate change output (output index 1 or later).
- In both cases, output index 0 **MUST** be the Taproot anchor output (P2TR scriptPubKey). No other output type is permitted at index 0.
- The operator **MUST NOT** consolidate the commit UTXO with any other UTXO while the commitment is pending reveal, as this would alter the ordinal assignment via FIFO.
- The commit transaction **SHOULD** have no more than one funding input when possible, to minimize FIFO complexity and make ordinal assignment verification trivial for validators.

5. Commit / Reveal Pattern

5.1 Overview

The Universal Bridge uses a two-phase commit-reveal scheme. Phase 1 publishes a Taproot UTXO encoding the commitment hash without revealing its content. Phase 2 spends that UTXO in a witness-carrying transaction that proves the pre-committed state. This scheme provides:

- **Front-running resistance:** The commitment payload is not visible on-chain until Phase 2. An adversary who observes the commit transaction cannot construct a valid competing commitment without knowing the internal key P and the full payload C.
- **Atomicity:** The commit and reveal are separate transactions. If the reveal never occurs, the commit UTXO is eventually recovered via the recovery script.
- **Ordinal binding:** The carrier sat's ordinal is locked to the commit UTXO from Phase 1 onward, establishing unambiguous provenance.



Taproot + Ordinal Commitment Specification

Universal Bridge (URIB)

Deterministic Commitment, Satoshi-Level Anchoring, and Cross-Rail Truth Binding

5.2 Commit Transaction Structure

The commit transaction **MUST** conform to the following structure. Any deviation renders the commitment non-compliant and validators **MAY** reject the associated reveal.

COMMIT TRANSACTION:

```
— nVersion: 2 (MUST be 2; nVersion=1
disallowed) nLockTime: 0 (no absolute timelock
on commit) vin_count: ≥1 vout_count: 2 or 3 (see below) Input[0]:
Funding UTXO —┐ Any type (P2WPKH, P2TR, P2WSH accepted) —┐ nSequence:
0xFFFFFFFF (BIP-125 RBF opt-in; MUST be set) —┐ Provides
sufficient sats for output[0] value + fees Output[0]: Taproot Anchor
Output —┐ CARRIER SAT LIVES HERE —┐ value: 546 sat
(common) OR 1 sat (rare+) —┐ scriptPubKey: 0x5120 || output_key (34
bytes, P2TR) —┐ output_key computed per Section 2.5 using bind_hash as
Merkle root h Output[1]: OP_RETURN Metadata (OPTIONAL but
RECOMMENDED) —┐ value: 0 sat (zero value;
unspendable) —┐ script: OP_RETURN (0x6a)
|| PUSH4(magic) (0x04 0x55 0x42 0x43 0x00) ||
PUSH1(version) (0x01 0x01) || PUSH32(bind_hash)
(0x20 || bind_hash[32]) —┐ Total script: 41 bytes (fits within standard
OP_RETURN 80-byte limit) —┐ This metadata enables lightweight
verification without parsing annex Output[2]: Change Output
(REQUIRED if change > 0) —┐ value: remaining sats after output[0] and
fees —┐ scriptPubKey: P2TR (RECOMMENDED) or P2WPKH —┐ MUST NOT be
output[0]'s script (no change-to-self)
```

5.3 Reveal Transaction Structure

The reveal transaction spends the commit UTUXO and publishes the full UBCCommitRecord, payload, and (for script-path reveals) the Tapscript witness elements.

REVEAL TRANSACTION:

```

-   nVersion: 2   nLockTime: <expiry_block_height if TIMELOCK flag set; else
0>               (expiry_block_height = ceil((record.expiry -
genesis_timestamp) / 600)
[approximate block height])   Input[0]: Spends COMMIT TX output[0]   L
txid:      commit_tx.txid   L   vout:      0   L   nSequence: 0xFFFFFFFFE
(BIP-125 RBF; MUST be set)   L   witness: (see witness stack below)
WITNESS STACK - KEY-PATH SPEND (KEY_PATH_ONLY flag set):   [0]: Schnorr
signature (64 bytes)   sig = BIP340_sign(tweaked_privkey, sighash)
sighash = taproot_sighash(REVEAL_TX, input_index=0, hash_type=0x00)
[annex]: annex_payload (see Section 5.5)   WITNESS STACK - SCRIPT-PATH SPEND
(TapTree present):   [0]: BIP-340 Schnorr signature (64 bytes)
over the script execution sighash   [1]: UB_COMMIT_SCRIPT bytes
(the leaf script being executed)   [2]: Control block (variable length)
= (leaf_version | parity_bit) [1 byte]   || internal_key [32 bytes]
|| merkle_path [0, 32, 64, or 96 bytes depending on tree depth]   For
3-leaf tree (depth 2 leaves): merkle_path = 64 bytes (2 siblings)   For
2-leaf tree (depth 1 leaves): merkle_path = 32 bytes (1 sibling)   [annex]:
annex_payload (see Section 5.5)   Output[0]: Destination output   L
value:      amount to deliver to recipient (in satoshis)   L
scriptPubKey: recipient's address (P2TR, P2WPKH, or P2WSH)   Output[1]:
Change output (OPTIONAL)   L   value:      remaining sats after output[0]
and fees   L   scriptPubKey: operator's change address

```

5.4 UB Commitment Script Template

When using the script-path variant, every Universal Bridge commitment leaf **MUST** use the following Tapscript template. Leaf version **MUST** be `0xC0`. All `<x>` placeholders are replaced with their actual byte values at commit-time construction.

```
UB_COMMIT_SCRIPT (leaf_version = 0xC0):

-   <operator_pubkey_32>          # PUSH 32 bytes: x-only operator public key
OP_CHECKSIG                     # 0xAC: verify Schnorr sig against
operator_pubkey OP_VERIFY       # 0x69: fail script if
CHECKSIG returned 0 <bind_hash_32> # PUSH 32 bytes: expected
bind_hash (Section 4.3) OP_DROP # 0x75: discard
bind_hash from stack (enforces it # by
inclusion; not evaluated dynamically) <expiry_block> # PUSH
<N> bytes: block height of expiry (CScriptNum) OP_CHECKLOCKTIMEVERIFY
# 0xB1: enforce nLockTime >= expiry_block #
(only included when TIMELOCK flag is set) OP_DROP #
0x75: discard expiry_block from stack OP_TRUE # 0x51:
push 1 (success)

- Non-timelocked variant (TIMELOCK flag NOT set): <operator_pubkey_32>
OP_CHECKSIG OP_VERIFY <bind_hash_32> OP_DROP OP_TRUE

- Notes:
  - The bind_hash is embedded as a script constant, not runtime-
    evaluated. This means a validator checking the script can confirm the
    expected bind_hash is hardcoded, making script substitution attacks
    impossible without knowing the original bind_hash.
  - operator_pubkey is the x-only (32-byte) form, consistent with BIP-342
    Tapscript OP_CHECKSIG semantics.
  - CScriptNum encoding for expiry_block: minimal push, big-
    endian, signed, as per Bitcoin Script integer serialization rules.
```

5.5 Annex Encoding

The witness annex is an optional final element in the witness stack, identified by the prefix byte `0x50` as defined in BIP-341. The Universal Bridge uses the annex to carry the full `UBCommitRecord` and raw payload bytes in the reveal transaction. Annex data is committed to by the BIP-341 sighash but is not validated by consensus rules beyond length limits.

annex_payload structure:					
– Offset	Len	Type	Description		
– 0	1	uint8	Annex marker: MUST be 0x50 (BIP-341)	1	1
uint8			UB annex version: MUST be 0x01 (this spec)	2	2
uint16_BE			record_len: MUST be 0x00A7 (167 decimal)	4	167 bytes
UBCommitRecord (fixed; see Section 4.1)	171	4	uint32_BE		
payload_len: number of bytes in payload	175	N	bytes		payload:
raw cross-chain state payload bytes					(N =
payload_len; maximum N = 520,000 bytes)					
– Total annex size: 175 + N bytes Maximum annex size: 520,175 bytes (imposed by Bitcoin witness stack element limit)					
– Constraints: annex[0] MUST be 0x50 annex[1] MUST be 0x01 for this specification record_len MUST be exactly 167 (0x00A7) payload_len MUST equal len(payload) exactly payload_len MUST NOT exceed 520,000 bytes					

Taproot + Ordinal Commitment Specification

Universal Bridge (URIB)

Deterministic Commitment, Satoshi-Level Anchoring, and Cross-Rail Truth Binding

NOTE — Annex Weight Accounting

Per BIP-341, witness annex bytes are counted at full witness weight (1 weight unit per byte) — they are

not

discounted like other witness data (which is 0.25 weight units per byte). For large payloads, the annex can dominate transaction fee cost. Implementers

SHOULD

compress payload bytes using DEFLATE before inclusion when

`payload_len > 4096`

, setting bit 7 of the UB annex version byte to indicate compression (e.g.,

`0x81`

= version 1, compressed).

5.6 Fee Estimation

The following reference formulas provide fee estimates. All sizes are in virtual bytes (vBytes); weight units (WU) are divided by 4 to obtain vBytes per Bitcoin's SegWit weight rules.

Transaction Type	Formula (vBytes)	Typical Value
Commit Tx (1 P2WPKH input, 2 P2TR outputs + OP_RETURN)	$10 + 41 \times n_{in} + 43 \times n_{out} + n_{witness}/4$	~210 vBytes
Reveal Tx — key-path spend (no annex)	$10.5 + 57.5 + 43 \times n_{out}$	~111 vBytes (1 output)
Reveal Tx — script-path, 2-leaf tree	$10.5 + 57.5 + \text{script_vb} + \text{ctrl_vb} + 43 \times n_{out}$	~200–350 vBytes
Annex overhead (per 1KB of payload)	$\text{ceil}(1024/4) = 256$ vBytes per KB	Scales linearly with payload size

```
Fee calculation:  fee_sat = ceil(tx_vbytes * fee_rate_sat_per_vbyte)
Recommended fee rate:  fee_rate = max(      min_relay_fee_rate,
# typically 1 sat/vbyte      mempool_fee_rate_75th_percentile      # from
local mempool stats      )      # Use the 75th percentile to target <30 minute
confirmation      # For time-sensitive reveals, use 90th percentile or higher
CPFP alternative:  If commit tx is stuck below fee floor, use CPFP on the
commit output      via a child transaction (not the reveal tx itself), to avoid
invalidating the ordinal binding.
```

5.7 RBF and Replace-by-Fee Policy

All Universal Bridge transactions use BIP-125 RBF opt-in to allow fee bumping in case of mempool congestion.

- All commit and reveal transactions **MUST** set `nSequence = 0xFFFFFFFF` on `input[0]`. This signals BIP-125 replaceability without disabling relative timelocks.
- If a commit transaction is stuck in the mempool, the operator **MAY** replace it with a higher-fee version, subject to these constraints:
 - `Output[0]` (the Taproot anchor) **MUST** be byte-for-byte identical in the replacement. Any change to the anchor output changes the `bind_hash` commitment and invalidates the operator's pre-signed `UBCommitRecord`.
 - The replacement **MAY** change the funding input, change output, and fee. It **MUST** include a fee rate increase of at least 1 sat/vbyte above the replaced transaction (BIP-125 Rule 4).
- A reveal transaction **MUST NOT** be replaced once the commit transaction has received at least 1 confirmation. Replacing a reveal after commit confirmation risks creating an ordinal assignment inconsistency if the replacement uses different inputs that alter FIFO ordinal routing.
- If a reveal transaction is stuck and the commit has zero confirmations, the operator **MAY** cancel both by double-spending the funding input of the commit, effectively abandoning the commitment. A new commitment with a fresh nonce **MUST** then be initiated.

6. Cross-Rail Anchoring Behavior

6.1 What Is Cross-Rail Anchoring?

In the Universal Bridge, a "rail" is any distinct blockchain network (Bitcoin, Ethereum, Solana, Cosmos, etc.). A cross-rail anchor is a Bitcoin Taproot UTXO whose commitment payload proves finalized state on two or more rails simultaneously. The `CROSS_RAIL` flag (bit 0) in `UBCommitRecord.flags` **MUST** be set for all cross-rail commitments.

Cross-rail anchoring provides a neutral settlement layer: since Bitcoin's proof-of-work chain has the highest accumulated work of any blockchain, anchoring state to it creates a trust-minimized notarization point that any rail can verify by running a Bitcoin light client or SPV proof system.

6.2 Multi-Rail Payload Structure

When the `CROSS_RAIL` flag is set, the commitment payload **MUST** conform to the following binary structure. This structure is serialized into the annex payload field (Section 5.5).

MultiRailPayload:			
Offset	Len	Field	Description
0	2	rail_count	Number of rails. Big-endian. MUST be ≥2, ≤16.
Per-Rail Entry (repeated rail_count times):			
+0	4	chain_id	UB chain registry ID (big-endian); see App.
+4	8	block_height	Finalized height on this rail (big-endian)
+12	32	state_root	Merkle/Patricia/state root at block_height
+44	32	tx_hash	Hash of the bridged transaction on this rail
+76	1	proof_type	0x00 = Merkle inclusion proof
			0x01 = ZK-SNARK proof
			0x02 = Optimistic
(challenge-period) proof	+77	4	proof_len
bytes (big-endian, uint32)	+81	proof_len	Length of proof
		proof	Proof bytes

Taproot + Ordinal Commitment Specification

Universal Bridge (URIB)

Deterministic Commitment, Satoshi-Level Anchoring, and Cross-Rail Truth Binding

(Merkle path, ZK proof, etc.)

— Total per-rail entry: 81 + proof_len bytes (variable) Constraints:
rail_count MUST be ≥ 2 and ≤ 16 chain_id MUST be a registered UB
chain ID (see Appendix B) block_height MUST be the finalized (not just
confirmed) height on that rail proof_type MUST be 0x00, 0x01, or 0x02;
other values are reserved proof_len MUST match len(proof) exactly

6.3 Anchor Finality Rules

Cross-rail anchoring finality depends on the confirmation requirements of both the Bitcoin anchor and the destination rail. The table below defines normative minimums. Implementations **MUST NOT** finalize state on the destination rail before the stated requirements are met.

Source Rail	Destination Rail	BTC Confirmations Required	Additional Requirement
Bitcoin	Ethereum L1	6	ETH block $\geq$ finalized (two-epoch finality checkpoint)
Bitcoin	EVM L2 — Optimistic (Arbitrum, Optimism)	100	L2 challenge period (7 days) fully expired; no valid fraud proof submitted
Bitcoin	EVM L2 — ZK (zkSync Era, Polygon zkEVM)	6	ZK validity proof verified and finalized on Ethereum L1
Bitcoin	Solana	6	SOL slot $\geq$ confirmed (≥ 31 slot supermajority)
Bitcoin	Cosmos Hub (via IBC)	6	IBC light client header updated on Cosmos Hub; $\geq 2/3$ validator set signed
Ethereum L1	Bitcoin	32 ETH blocks (~6.4 min)	ETH source finalized + BTC anchor UTXO mined with ≥ 1 confirmation
Solana	Bitcoin	1 (BTC anchor mined)	150 SOL slots (~60s) confirmed on Solana prior to BTC broadcast
Cosmos Hub	Bitcoin	1 (BTC anchor mined)	IBC packet committed on Cosmos Hub with $\geq 2/3$ validators; header relayed

6.4 Anchor Chain of Custody

To maintain end-to-end auditability, every cross-rail anchor **MUST** cryptographically reference its predecessor anchor in the same bridge session. This forms a tamper-evident linked list of anchors — any break in the chain is immediately detectable.

```
chain_link_hash = SHA256(      prev_anchor_txid    [32 bytes, big-endian txid
of predecessor commit tx] ||      prev_anchor_vout    [4 bytes, little-endian
vout index of predecessor]      ||      chain_id_src      [4 bytes, big-endian]
||      chain_id_dst      [4 bytes, big-endian]
||      this_nonce      [8 bytes, big-endian; from UBCommitRecord.nonce] )
# Result: 32 bytes chain_link_hash MUST appear as the FIRST 32 bytes of
payload_hash in UBCommitRecord. That is:      payload_hash =
SHA256(chain_link_hash || actual_payload_bytes) This design ensures: - The
payload_hash field simultaneously commits to the chain of custody (via
chain_link_hash) and to the payload content (via hashing). - Validators can
verify the chain link without access to the full payload by checking that
SHA256(chain_link_hash || anything) == record.payload_hash after
extracting chain_link_hash from the reveal annex. For the FIRST anchor in a
bridge session (no predecessor):      prev_anchor_txid = 0x00...00 (32 zero
bytes)      prev_anchor_vout = 0xFFFFFFFF (sentinel value)
```

6.5 Anchor Reorg Handling

Bitcoin reorgs invalidate any commitment whose commit transaction was in the orphaned chain tip. The Universal Bridge operator's node **MUST** implement the following reorg detection and recovery procedure.

- The operator node **MUST** maintain a rolling window of the last 100 block headers. If a reorganization is detected (a new chain tip with more accumulated work that does not include a previously confirmed block), all commitments whose commit transactions were in orphaned blocks **MUST** be marked INVALIDATED.
- Upon detecting a reorg that invalidates a commitment, the operator **MUST**:

Set the affected commitment state to INVALIDATED in the bridge state machine (Section 6.6).

If the underlying payload is still valid and has not expired: re-broadcast the commit transaction with a fresh nonce to avoid txid collision. A new nonce requires a new UBCommitRecord with an updated chain_link_hash referencing the last valid predecessor anchor.

Notify all counterparty rails that the prior anchor is invalid. Each rail's governance or IBC channel handling **MUST** roll back any state tentatively finalized on that anchor.

- Sats that were "in flight" during a reorg — i.e., the carrier sat's UTXO was in an orphaned block — retain their ordinal. If the same sat subsequently appears in a UTXO after the reorg (either because the transaction was re-mined or the inputs were respent), the ordinal binding is still valid and the protocol **MAY** proceed with the original or a re-issued reveal transaction.

Taproot + Ordinal Commitment Specification

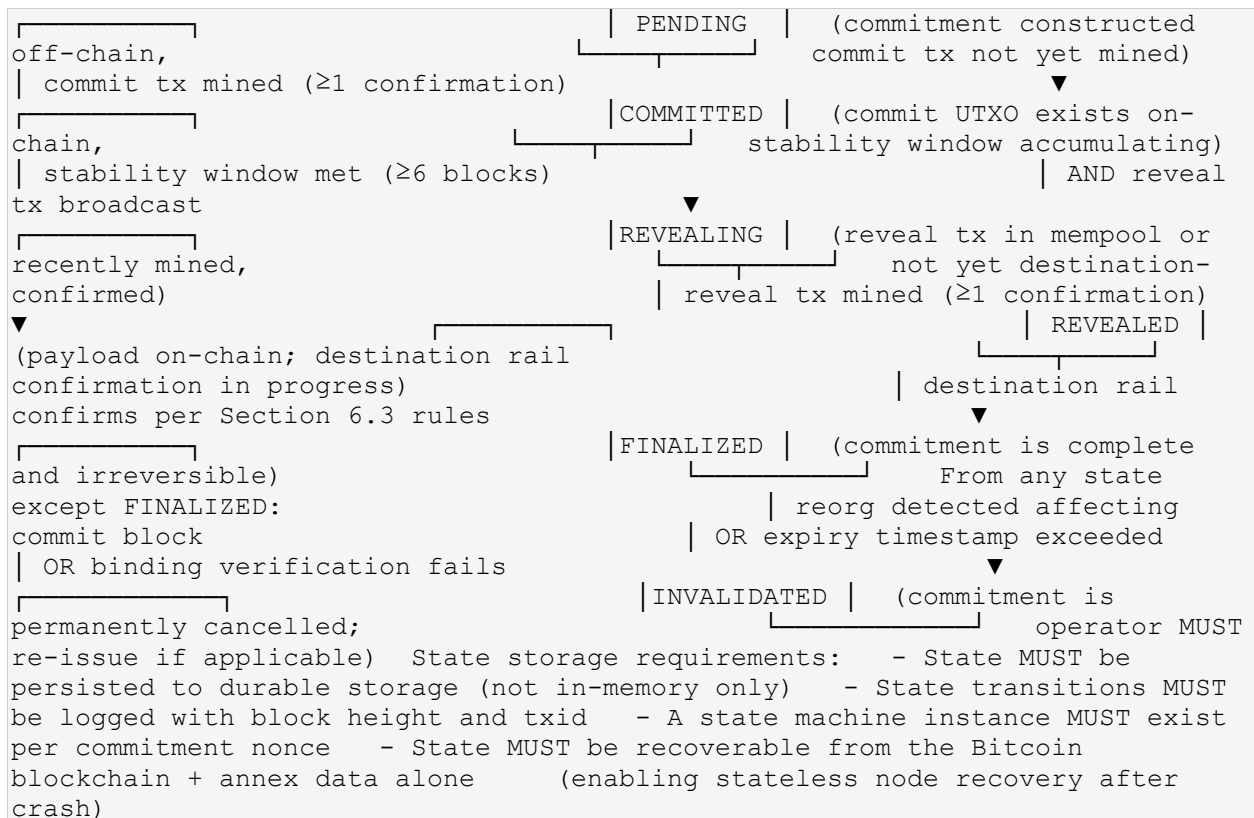
Universal Bridge (URIB)

Deterministic Commitment, Satoshi-Level Anchoring, and Cross-Rail Truth Binding

- The operator **SHOULD** subscribe to hashblock notifications via Bitcoin ZMQ and compare each new block header's previousblockhash field against the locally stored chain to detect reorgs within one block confirmation latency.

6.6 Cross-Rail Finality State Machine

The Universal Bridge defines a normative state machine for each commitment. Implementations **MUST** maintain this state for every active commitment and **MUST** transition states only on the defined events.



6.7 Emergency Recovery Script

Every Universal Bridge commitment that uses the script-path variant (TapTree present) **MUST** include a Recovery leaf in the TapTree. This provides an emergency exit for the bridge operator if the reveal cannot be completed before the commitment expires.

```
RECOVERY_SCRIPT (leaf_version = 0xC0):  
  
- <operator_pubkey_32>          # x-only 32-byte operator pubkey  OP_CHECKSIG  
# 0xAC  OP_VERIFY                # 0x69  <recovery_timelock>          #  
CScriptNum: expiry_block + RECOVERY_TIMELOCK_DELTA  
# RECOVERY_TIMELOCK_DELTA = 1008 blocks (~1 week)  OP_CHECKLOCKTIMEVERIFY  
# 0xB1: enforce nLockTime ≥ recovery_timelock  OP_DROP              #  
0x75  OP_TRUE                    # 0x51  
  
- recovery_timelock = record.expiry_block + 1008  Properties:  - Recovery  
can ONLY be triggered AFTER expiry_block + 1008 blocks.      This ensures the  
normal reveal window is fully exhausted before              emergency recovery becomes  
possible.  - Only the operator's key (same pubkey as in  
UBCommitRecord.operator_pubkey) can execute the recovery script.  -  
Recovery MUST NOT be used to steal user funds before expiry. The 1008-block  
delta provides approximately one week of grace period for dispute resolution  
before the operator can unilaterally recover the UTXO.  - The carrier sat is  
recovered along with the UTXO value in the recovery      transaction. The  
sat's ordinal binding to the payload becomes null and void  upon recovery  
(the commitment is treated as INVALIDATED).
```

7. Implementation Reference

7.1 Canonical Parameter Table

All protocol constants are defined below. Implementations **MUST** use these exact values. No parameter is implementation-defined.

Parameter Name	Value	Type	Description
MAGIC	0x55424300	uint32_BE	UBCommitRecord magic bytes ("UBC\x00")
PROTOCOL_VERSION	0x01	uint8	Current specification version byte
LEAF_VERSION	0xC0	uint8	Tapscript leaf version for all UB tap-leaves (192 decimal)
ANNEX_MARKER	0x50	uint8	BIP-341 annex identification prefix byte
UB_ANNEX_VERSION	0x01	uint8	UB-specific annex version; byte 1 of annex payload
DUST_LIMIT_COMMON	546	uint64 (satoshis)	Minimum output value for common sat carrier (P2TR dust limit)
DUST_LIMIT_RARE	1	uint64 (satoshis)	Minimum output value for rare+ sat carrier (1 sat minimal carry)
STABILITY_WINDOW	6	uint32 (blocks)	Minimum confirmations on commit UTXO before reveal tx broadcast
REORG_WINDOW	100	uint32 (blocks)	Rolling window depth for reorg detection monitoring
MAX_RAILS	16	uint8	Maximum number of rails in a MultiRailPayload
RECOVERY_TIMELOCK_DELTA	1008	uint32 (blocks)	Additional blocks after expiry_block before recovery is valid (~1 week)

Taproot + Ordinal Commitment Specification

Universal Bridge (URIB)

Deterministic Commitment, Satoshi-Level Anchoring, and Cross-Rail Truth Binding

Parameter Name	Value	Type	Description
NONCE_SIZE	8	uint8 (bytes)	Byte length of UBCCommitRecord.nonce field
RECORD_SIZE	167	uint16 (bytes)	Fixed total byte length of UBCCommitRecord
MAX_PAYLOAD_SIZE	520000	uint32 (bytes)	Maximum raw payload length in annex (Bitcoin witness stack limit)
HIGH_VALUE_THRESHOLD	1000000000	uint64 (satoshis)	10 BTC threshold above which 24-block confirmation requirement applies
RBF_SEQUENCE	0xFFFFFFFF	uint32	nSequence value signaling BIP-125 RBF opt-in
FIRST_SENTINEL_TXID	0x00...00 (32 bytes)	bytes32	Sentinel prev_anchor_txid for the first anchor in a session
FIRST_SENTINEL_VOUT	0xFFFFFFFF	uint32_LE	Sentinel prev_anchor_vout for the first anchor in a session

7.2 Tagged Hash Domains

The following table lists all tagged hash domains used in this specification. The double-SHA256 tag expansion ensures each domain is cryptographically isolated.

Tag String (UTF-8)	SHA256(tag) (hex, first 8 bytes shown)	Usage / Section
"TapTweak"	e80fe1... (BIP-341 defined)	Taproot key tweak scalar derivation (§2.5, §2.6)
"TapLeaf"	aeea8f... (BIP-341 defined)	Tapleaf hash construction (§2.2)
"TapBranch"	14b0bb... (BIP-341 defined)	Tapbranch hash construction (§2.3)
"TapSighash"	(BIP-341 defined)	Taproot sighash (used internally by signing; not extended by UB)
"UB/Commit/v1"	Implementer-computed	Universal Bridge payload commitment hash H_UBCommit (§2.6)
"UB/OrdBind/v1"	Implementer-computed	Ordinal satoshi binding hash H_UBOrdBind (§4.3)

```
Expansion of tagged_hash for "UB/Commit/v1":  tag_bytes = b"UB/Commit/v1"
# 12 bytes UTF-8    tag_hash = SHA256(tag_bytes)           # 32 bytes
prefix    = tag_hash || tag_hash                        # 64 bytes (double prefix per BIP-
340)    result    = SHA256(prefix || message)           # 32-byte output  Same
construction for "UB/OrdBind/v1" with tag_bytes = b"UB/OrdBind/v1" (13 bytes)
```

Taproot + Ordinal Commitment Specification

Universal Bridge (URIB)

Deterministic Commitment, Satoshi-Level Anchoring, and Cross-Rail Truth Binding

7.3 Test Vectors

The following worked examples are provided for implementation validation. Both examples use deterministic fictional values. Implementations **SHOULD** validate against these vectors before deployment.

7.3.1 Example 1 — Key-Path Only Commitment (No Script Tree)

Scenario: Bridge operator anchors a finalized ETH→BTC withdrawal of 1.5 ETH from Ethereum Mainnet (chain 0x00000064) to Bitcoin Mainnet (0x00000001). The payload is a serialized EIP-712 withdrawal proof struct. No TapScript tree; key-path spend only.

```
— INPUTS —
Internal Key P (bridge operator MuSig2 aggregate, hex, x-only 32 bytes): P
= 79BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B16F81798 (This is
the x-coordinate of secp256k1 generator G itself, used here for
illustrative purposes only. Production keys MUST be unique.) Payload C
(human-readable description): ETH Mainnet → BTC Mainnet withdrawal From:
0xAbCdEf1234567890AbCdEf1234567890AbCdEf12 (Ethereum address) To:
bclqxy2kgdygjrsqtzq2n0yrf2493p83kkfjhx0wlh (Bitcoin address) Amount: 1.5
ETH (1,500,000,000,000,000,000 wei = 0x14D1120D7B160000) ETH Tx:
0xABCD1234...DEAD5678 (finalized, slot 9,500,000) Nonce:
0x0102030405060708 — COMPUTATION

Step 1:
Compute SHA256 of raw payload bytes C: payload_hash = SHA256(C) =
3A7BD3E2360A3D29EEA3F0CA8B31EF99E79D9A9C4E75A7D9B5F2E1C6D4A8B2F Step 2:
Compute bind_hash via H_UBOrdBind: sat_ordinal = 0x0001312D00000000
(example uncommon sat, block 1,250,000) chain_id_src = 0x00000064
(Ethereum Mainnet) chain_id_dst = 0x00000001 (Bitcoin Mainnet)
payload_hash = 3A7BD3E2...8B2F (from Step 1) timestamp =
0x66C4E000 (Unix: 1,724,000,000 ≈ Aug 2024) bind_hash =
H_UBOrdBind( 0x0001312D00000000 || 0x00000064 ||
0x00000001 ||
3A7BD3E2360A3D29EEA3F0CA8B31EF99E79D9A9C4E75A7D9B5F2E1C6D4A8B2F ||
0x66C4E000 ) bind_hash ≈
B7C4A918E3D52F6A0145C89B3F7A2DE4C61F09823B45D7E61A2093F5C84B17A
(Illustrative; compute
SHA256(SHA256("UB/OrdBind/v1") || SHA256("UB/OrdBind/v1") || input)) Step 3:
Compute tweak scalar t: t = int(H_TapTweak(bytes(P) || bind_hash)) t =
int(SHA256(SHA256("TapTweak") || SHA256("TapTweak") || P || bind_hash)) t ≈
4F2A18C7...E3D9 (32 bytes interpreted as integer mod n) Verify: t < n ✓
(virtually guaranteed) Step 4: Compute output key Q: Q = lift_x(int(P)) +
t · G Q_x ≈ A3F81B2C9D4E5F607182930A4B5C6D7E8F90A1B2C3D4E5F6071829A0B1C2D3E
(x-coordinate of the tweaked point) If has_even_y(Q): output_key = Q_x,
parity_bit = 0 If odd Y: output_key = Q_x, parity_bit = 1 (Q is
implicitly negated) Assume: parity_bit = 0 for this example Step 5:
```

Taproot + Ordinal Commitment Specification

Universal Bridge (URIB)

Deterministic Commitment, Satoshi-Level Anchoring, and Cross-Rail Truth Binding

```
scriptPubKey:  scriptPubKey = 0x5120 ||
A3F81B2C9D4E5F607182930A4B5C6D7E8F90A1B2C3D4E5F6071829A0B1C2D3E  =
5120A3F81B2C9D4E5F607182930A4B5C6D7E8F90A1B2C3D4E5F6071829A0B1C2D3E  (34
bytes total)  — COMMIT TX OUTPUT STRUCTURE
Output[0]:  1 sat (uncommon
rarity carrier)  scriptPubKey: 5120A3F81B2C...2D3E  (P2TR)
Output[1]:  0 sat OP_RETURN: 6a04554243000166B7C4A918...17A (optional
metadata) Output[2]:  change (variable) P2TR change address
```

7.3.2 Example 2 — Script-Path Commitment (With TapTree, NUMS Internal Key)

Scenario: An NFT cross-rail bridge using a script-path commitment with a 2-leaf TapTree: a UB commit leaf and a recovery leaf. Internal key is the UB NUMS point (no key-path spend possible).

```
— INPUTS —
NUMS Internal Key P:  P_NUMS = lift_x(SHA256("UB/NUMS/internal-key/v1"))
SHA256("UB/NUMS/internal-key/v1") =  E3D5A7F2... (32 bytes; implementer
must compute; first iteration assumed valid)  P_NUMS =
E3D5A7F2184B6C9D3E5F7A8B2C1D4E6F7A8B9C0D1E2F3A4B5C6D7E8F9A0B1C2  bind_hash
(from Section 4.3, computed as in Example 1, with NFT payload):  bind_hash =
C2A14F8B3D76E90521C4B7A8F6E5D3C2B1A09F8E7D6C5B4A3F2E1D0C9B8A7F6
operator_pubkey (33-byte compressed for CHECKSIG in TapScript):  op_pk =
02D1E2F3A4B5C6D7E8F9A0B1C2D3E4F5A6B7C8D9E0F1A2B3C4D5E6F7A8B9C0D1
expiry_block = 850000  (CScriptNum: 0x0CF850 → 3-byte push 0x03 0x50 0xF8
0x0C)  — LEAF HASH COMPUTATIONS
UB_COMMIT_SCRIPT bytes
(non-timelocked variant, 39 bytes):  20 D1E2F3...C0D1  # OP_PUSH32
operator_pubkey_x32  AC  # OP_CHECKSIG  69
# OP_VERIFY  20 C2A14F...A7F6  # OP_PUSH32 bind_hash  75
# OP_DROP  51  # OP_TRUE  Leaf_A hash = H_TapLeaf(0xC0 ||
compact_size(39) || UB_COMMIT_SCRIPT)  = H_TapLeaf(C0 27
20D1...AC6920C2...7551)  Leaf_A =
8F3C1A72B9E4D56F0A1B2C3D4E5F6A7B8C9D0E1F2A3B4C5D6E7F8A9B0C1D2E3
RECOVERY_SCRIPT bytes (37 bytes):  20 D1E2F3...C0D1  # OP_PUSH32
operator_pubkey_x32  AC  # OP_CHECKSIG  69
# OP_VERIFY  03 50F80C  # OP_PUSH3 expiry_block+1008 CScriptNum
B1  # OP_CHECKLOCKTIMEVERIFY  75  #
OP_DROP  51  # OP_TRUE  Leaf_B hash = H_TapLeaf(0xC0 ||
compact_size(37) || RECOVERY_SCRIPT)  Leaf_B =
2D4E6F8A0B1C2D3E4F5A6B7C8D9E0F1A2B3C4D5E6F7A8B9C0D1E2F3A4B5C6D7  —
TAPBRANCH (ROOT)
Compare Leaf_A vs Leaf_B lexicographically:  8F3C1A... > 2D4E6F... →
Leaf_B comes first  Merkle Root h = H_TapBranch(Leaf_B || Leaf_A)  h =
tapbranch_hash(Leaf_B, Leaf_A)  [Leaf_B ≤ Leaf_A is FALSE; so sort]  h =
H_TapBranch(2D4E6F...D7 || 8F3C1A...E3)  h =
5C7E9A1B2D4F6A8B0C1D2E3F4A5B6C7D8E9F0A1B2C3D4E5F6A7B8C9D0E1F2A3  — TAPROOT
TWEAK  t =
int(H_TapTweak(P_NUMS || h))  = int(H_TapTweak(E3D5A7...C2 || 5C7E9A...A3))
t = 1A2B3C4D5E6F7A8B9C0D1E2F3A4B5C6D7E8F9A0B1C2D3E4F5A6B7C8D9E0F1A2  Verify:
t < n  ✓  Q = P_NUMS + t·G  Q_x =
```

Taproot + Ordinal Commitment Specification

Universal Bridge (URIB)

Deterministic Commitment, Satoshi-Level Anchoring, and Cross-Rail Truth Binding

```
7B8C9D0E1F2A3B4C5D6E7F8A9B0C1D2E3F4A5B6C7D8E9F0A1B2C3D4E5F6A7B8    Assume
has_even_y(Q): parity_bit = 0    scriptPubKey = 5120
7B8C9D0E1F2A3B4C5D6E7F8A9B0C1D2E3F4A5B6C7D8E9F0A1B2C3D4E5F6A7B8    — CONTROL
BLOCK (for Leaf_A / UB_COMMIT_SCRIPT spend) —————
control_block = (leaf_version | parity_bit) # 0xC0 | 0x00 = 0xC0
|| P_NUMS                                # 32 bytes internal key          ||
Leaf_B                                  # 32 bytes sibling hash (Merkle proof)
control_block = C0                      ||
E3D5A7F2184B6C9D3E5F7A8B2C1D4E6F7A8B9C0D1E2F3A4B5C6D7E8F9A0B1C2
|| 2D4E6F8A0B1C2D3E4F5A6B7C8D9E0F1A2B3C4D5E6F7A8B9C0D1E2F3A4B5C6D7    Total: 1
+ 32 + 32 = 65 bytes    — REVEAL WITNESS STACK (script-path)
————— [0]: Schnorr signature (64 bytes)
over script execution sighash [1]: UB_COMMIT_SCRIPT bytes (39 bytes) [2]:
control_block (65 bytes) [annex]: 50 01 00A7 <UBCommitRecord 167 bytes>
<payload_len 4 bytes> <payload>
```

7.4 Security Considerations

7.4.1 Malleability Prevention

BIP-340 Schnorr signatures are non-malleable by construction when properly implemented. Specific properties that Universal Bridge implementers must be aware of:

- **X-only key encoding:** By committing only the x-coordinate to the scriptPubKey and normalizing all public keys to even Y before hashing, the protocol eliminates the ambiguity inherent in full compressed pubkeys (which have two possible Y values). Any attempt to substitute the negated key would produce a different x-coordinate, failing verification.
- **Even-Y normalization in tweak:** The output key is always the even-Y representative of Q. Signers **MUST** negate their private key if the computed Q has odd Y, ensuring the signing and verification keys are consistent. Failure to negate causes signature verification failure.
- **Strict DER vs. BIP-340:** Unlike ECDSA, Schnorr signatures under BIP-340 are not subject to ECDSA DER malleability. There is exactly one valid signature for a given (key, message, nonce) triple. Implementers **MUST NOT** use ECDSA for any UB signing operation.

Taproot + Ordinal Commitment Specification

Universal Bridge (URIB)

Deterministic Commitment, Satoshi-Level Anchoring, and Cross-Rail Truth Binding

7.4.2 Sat Grinding Attack

An adversary who controls the funding inputs of the commit transaction could attempt to construct inputs such that a different ordinal — not the intended carrier — is FIFO-assigned to output index 0, subverting the binding. Mitigations:

- Validators **MUST** perform a full FIFO trace from the commit transaction's inputs to verify that `UBCommitRecord.sat_ordinal` is among the ordinals assigned to output index 0.
- Operators **SHOULD** use exactly one funding input when possible. With a single input, the FIFO trace is trivially verifiable.
- When multiple inputs are required, the operator **MUST** ensure the carrier sat arrives via `input[0]` and is within the first `output[0].value` sats of `input[0]`'s ordinal sequence.

7.4.3 Tweak Collision Security

A collision in `H_TapTweak` would allow an adversary to construct a different script tree that produces the same output key Q. The security argument:

- `H_TapTweak` is modeled as a random oracle. Finding a collision requires 2^{128} hash evaluations (birthday bound on a 256-bit hash), which is computationally infeasible with current and foreseeable technology.
- The UB-specific tagged hashes (`H_UBCommit`, `H_UBOrdBind`) add an additional domain separation layer, ensuring that even if a generic SHA256 attack is found, it does not immediately translate to a UB-domain attack.
- The `tagged_hash` construction's double-prefix (`SHA256(tag) || SHA256(tag)`) ensures that the hash domain is determined at key setup time, not at evaluation time, preventing length-extension attacks on the tag prefix.

Taproot + Ordinal Commitment Specification

Universal Bridge (URIB)

Deterministic Commitment, Satoshi-Level Anchoring, and Cross-Rail Truth Binding

7.4.4 Dust Attack

7.4.5 Front-Running Prevention

The commit phase hides payload content entirely. From the commit transaction, a blockchain observer learns only:

- That a P2TR output was created (indistinguishable from any other Taproot output in key-path mode).
- Optionally (if OP_RETURN is present), the `bind_hash` — which commits to the payload but does not reveal it.

An adversary who observes only `bind_hash` cannot construct a valid competing UBCCommitRecord without knowing the internal key P and the full payload C, since:

```
bind_hash = H_UBOrdBind(sat_ordinal || chain_id_src || chain_id_dst ||
payload_hash || timestamp)    # payload_hash = SHA256(C)    # Without C, an
adversary cannot find a payload_hash that makes    # their own bind_hash match
the observed bind_hash (SHA256 preimage resistance).    # Without P, they
cannot reproduce output_key Q from bind_hash.
```

7.4.6 Reorg Attack

Sections 3.5 and 6.5 define the normative reorg handling procedures. The 6-block stability window provides protection against shallow reorgs (depth 1–5). The 100-block reorg monitoring window covers the historical maximum observed Bitcoin reorg depth (which has never exceeded depth 7 on mainnet post-2013) with a margin exceeding 10×. For the highest-value transfers, requiring 100 confirmations before destination-rail finalization provides security equivalent to approximately six hours of honest miner mining, far exceeding the economic cost of any conceivable reorg on mainnet.

7.5 Compliance Checklist

The following normative requirements are extracted from all sections of this specification. QA engineers MUST verify each item before signing off on a release. Each item references the governing section.

[] 1. (§1.1) All tagged_hash computations use UTF-8 tag bytes with double-SHA256 prefix. No reuse of existing BIP tag strings for new UB hash domains.

[] 2. (§2.1) Internal key P is a valid x-only secp256k1 public key (32 bytes). P is never the point at infinity.

[] 3. (§2.1) When key-path spend is disabled, P is the canonical UB NUMS point derived from SHA256("UB/NUMS/internal-key/v1") with counter suffix if needed.

[] 4. (§2.1) P is not reused across distinct commitment nonces without explicit key rotation.

[] 5. (§2.2) All UB tap-leaves use leaf_version = 0xC0. No other leaf version is used.

Taproot + Ordinal Commitment Specification

Universal Bridge (URIB)

Deterministic Commitment, Satoshi-Level Anchoring, and Cross-Rail Truth Binding

[] 6. (§2.5) Tweak scalar t satisfies t

$<$

n before computing Q . Invalid tweaks are rejected (script tree must be changed).

[] 7. (§2.5) Output key is always normalized to even- Y (negate Q if `has_odd_y(Q)`).

Parity bit is tracked for control block construction.

[] 8. (§2.5) `scriptPubKey` is exactly 34 bytes: `0x5120 || output_key[32]`.

[] 9. (§3.2) Ordinal assignment is validated by a full FIFO trace from the commit transaction's inputs to `output[0]`. `sat_ordinal` MUST be in `output[0]`'s assigned ordinal set.

[] 10. (§3.4) When `RARITY_REQUIRED` flag is set, the carrier sat rarity is verified to be $\geq$ `uncommon` before committing.

[] 11. (§3.4) If required rarity sat is unavailable, commitment is deferred — lower-rarity substitution is prohibited.

Taproot + Ordinal Commitment Specification

Universal Bridge (URIB)

Deterministic Commitment, Satoshi-Level Anchoring, and Cross-Rail Truth Binding

[] 12. (§3.5) Commit UTXO is held for $\geq$ STABILITY_WINDOW (6) blocks before reveal transaction broadcast.

[] 13. (§4.1) UBCCommitRecord magic field equals exactly 0x55424300. Version field equals 0x01. Reserved flag bits 5–7 are zero.

[] 14. (§4.1) UBCCommitRecord.signature is a valid BIP-340 Schnorr signature by operator_pubkey over SHA256(record[0:103]).

[] 15. (§4.1) expiry > timestamp in UBCCommitRecord (expiry is strictly in the future at commit time).

[] 16. (§4.4) Binding verification steps 1–11 are all executed and all assertions must pass for VALID result. No step may be skipped.

[] 17. (§4.5) Commit output[0] value is exactly 546 sat for common carriers, exactly 1 sat for rare+ carriers. No other values are accepted.

Taproot + Ordinal Commitment Specification

Universal Bridge (URIB)

Deterministic Commitment, Satoshi-Level Anchoring, and Cross-Rail Truth Binding

[] 18. (§5.2) Commit transaction uses nVersion=2. input[0] nSequence = 0xFFFFFFFF (RBF opt-in). Output[0] is the P2TR anchor.

[] 19. (§5.3) Reveal transaction witness annex begins with 0x50 (annex marker) and UB annex version 0x01.

[] 20. (§5.4) UB_COMMIT_SCRIPT uses leaf_version 0xC0 and embeds bind_hash as a static 32-byte constant. OP_CHECKLOCKTIMEVERIFY is included iff TIMELOCK flag is set.

[] 21. (§5.5) annex payload_len matches the actual payload byte length exactly. Payload does not exceed 520,000 bytes.

[] 22. (§5.7) Reveal transactions are not replaced once the commit transaction has ≥ 1 confirmation.

Taproot + Ordinal Commitment Specification

Universal Bridge (URIB)

Deterministic Commitment, Satoshi-Level Anchoring, and Cross-Rail Truth Binding

[] 23. (§6.2) When CROSS_RAIL flag is set, payload conforms to MultiRailPayload structure. rail_count is between 2 and 16 inclusive.

[] 24. (§6.3) Destination rail is not finalized before the required Bitcoin confirmation count from the anchor finality table.

[] 25. (§6.4) chain_link_hash is the first 32 bytes of payload_hash, correctly linking to the predecessor anchor (or sentinel values for session-first anchors).

[] 26. (§6.5) Operator node monitors a rolling 100-block window for reorgs. Affected commitments are marked INVALIDATED within one block of reorg detection.

[] 27. (§6.6) Commitment state machine persists to durable storage. Transitions are logged with block height and txid. State is recoverable from blockchain data alone.

[] 28. (§6.7) Every script-path TapTree includes a Recovery leaf. Recovery timelock = expiry_block + 1008 blocks.

Taproot + Ordinal Commitment Specification

Universal Bridge (URIB)

Deterministic Commitment, Satoshi-Level Anchoring, and Cross-Rail Truth Binding

[] 29. (§7.1) All protocol constants match the canonical parameter table exactly. No implementation-defined overrides.

[] 30. (§7.4) Signing uses BIP-340 Schnorr exclusively. ECDSA is not used for any UB signing operation.

Appendix A — BIP-341 Reference Pseudocode (Condensed)

The following pseudocode is condensed from BIP-341 for implementer reference. The Universal Bridge extensions are annotated with [UB EXTENSION] comments.

```

—— taproot_tweak_pubkey (BIP-341, $Spending conditions)
    taproot_tweak_pubkey(pubkey, h):          # pubkey: 32-byte
x-only internal key      # h:      32-byte Merkle root (or H_UBCommit output
for keypath-only mode)   # [UB EXTENSION]: h may be bind_hash from
H_UBOrdBind             t = int_from_bytes(tagged_hash("TapTweak", pubkey + h))
if t >= SECP256K1_ORDER: raise ValueError("Tweak out of range")      P
= lift_x(int_from_bytes(pubkey))      Q = point_add(P, point_mul(G, t))
parity = 0 if has_even_y(Q) else 1      return (bytes_from_int(x(Q)), parity)
# Returns (output_key_32_bytes, parity_bit) —— taproot_tweak_seckey (BIP-
341, signing)
    taproot_tweak_seckey(seckey,
h):      # seckey: 32-byte secret key (scalar)      # h:      32-byte Merkle
root      P = point_mul(G, int_from_bytes(seckey))      if not has_even_y(P):
seckey = SECP256K1_ORDER - int_from_bytes(seckey)      # negate if P has odd Y
t = int_from_bytes(tagged_hash("TapTweak", bytes_from_int(x(P)) + h))      if
t >= SECP256K1_ORDER: raise ValueError("Tweak out of range")
tweaked = (int_from_bytes(seckey) + t) % SECP256K1_ORDER      return
bytes_from_int(tweaked)      # The tweaked secret key signs for Q; caller must
additionally negate if Q odd Y      # [UB EXTENSION]: After computing tweaked
seckey, re-check has_even_y(Q).      # If Q has odd Y, the signing key must be
negated once more (negate twice = identity      # if both P and Q have odd Y;
follow BIP-340 $Signing exactly). —— control_block_construction (BIP-341)

    control_block(leaf_version, parity_bit,
internal_pubkey, merkle_path):      # leaf_version: uint8, 0xC0 for UB leaves
# parity_bit: 0 or 1 (from taproot_tweak_pubkey)      # internal_pubkey: 32-
byte x-only key P      # merkle_path: concatenation of 32-byte sibling hashes
along the path from      # the leaf to the root (ordered leaf-to-
root)      first_byte = (leaf_version & 0xFE) | parity_bit      return
bytes([first_byte]) + internal_pubkey + merkle_path      # [UB EXTENSION]:
merkle_path for a 2-leaf tree = 1 sibling hash = 32 bytes      # [UB
EXTENSION]: merkle_path for a 3-leaf tree (depth 2) = 2 hashes = 64 bytes      #
(the UB 3-leaf tree has one leaf at depth 1 and two at depth 2;      #
the depth-2 leaves need 2 sibling hashes in their proof path) ——

taproot_sighash (BIP-341, Signature validation)
    taproot_sighash(tx, input_index, hash_type, spent_utxos):      # hash_type
0x00 = SIGHASH_DEFAULT (= SIGHASH_ALL for Taproot)      # [UB EXTENSION]:
Always use hash_type = 0x00 for UB reveal transactions      #
unless a specific SIGHASH variant is required by protocol policy.      # Full
sighash algorithm per BIP-341 $Common signature message; not reproduced      #
here in full. Implementers MUST use a BIP-341-compliant library.      ... ——
UB Extension: keypath-only variant

    UB_taproot_tweak_pubkey_keypath(pubkey, payload_C):      # [UB EXTENSION
$2.6]: Replace h with H_UBCommit(C) for keypath-only mode      h_ub =
tagged_hash("UB/Commit/v1", payload_C)      return
taproot_tweak_pubkey(pubkey, h_ub)      # Otherwise identical to standard
taproot_tweak_pubkey UB_taproot_tweak_pubkey_binding(pubkey, record):      #

```

```
[UB EXTENSION $4.3]: Replace h with bind_hash for sat-binding mode
bind_hash = tagged_hash("UB/OrdBind/v1",
+               record.sat_ordinal_bytes8_BE
+               record.chain_id_src_bytes4_BE +
+               record.chain_id_dst_bytes4_BE +
+               record.payload_hash +
+               record.timestamp_bytes4_BE )
return taproot_tweak_pubkey(pubkey,
bind_hash)
```

Appendix B — Chain ID Registry (Universal Bridge)

The UB chain ID is a 4-byte big-endian identifier used in `UBCommitRecord.chain_id_src`, `chain_id_dst`, and `MultiRailPayload.chain_id` fields. The registry below defines all currently recognized chains. New chains **MUST** be registered with the Universal Bridge Protocol Working Group before use; unregistered chain IDs are invalid and validators **MUST** reject commitments referencing them.

Chain Name	UB chain_id (hex)	Underlying Protocol ID	Notes
Bitcoin Mainnet	0x00000001	—	Primary anchoring chain for all UB commitments
Bitcoin Testnet4	0x00000002	—	Bitcoin Testnet4 (2024+); not for production use
Bitcoin Signet	0x00000003	—	Bitcoin Signet; development/testing only
Ethereum Mainnet	0x00000064	EIP-155 chain ID 1 (decimal 100 hex)	Ethereum L1 post-Merge
Ethereum Sepolia	0x00000065	EIP-155 chain ID 11155111	Ethereum Sepolia testnet
Ethereum Holesky	0x00000066	EIP-155 chain ID 17000	Ethereum Holesky testnet
Polygon PoS	0x00000089	EIP-155 chain ID 137 (decimal 137 = 0x89)	Polygon PoS Mainnet

Taproot + Ordinal Commitment Specification

Universal Bridge (URIB)

Deterministic Commitment, Satoshi-Level Anchoring, and Cross-Rail Truth Binding

Chain Name	UB chain_id (hex)	Underlying Protocol ID	Notes
Arbitrum One	0x0000A4B1	EIP-155 chain ID 42161 (0xA4B1)	Arbitrum One L2 (Optimistic); 100-block BTC confirmation required
Arbitrum Sepolia	0x066EEE91	EIP-155 chain ID 421614	Arbitrum Sepolia testnet
Base	0x00002105	EIP-155 chain ID 8453 (0x2105)	Base L2 (Optimistic Rollup on Ethereum); 100-block BTC confirmation required
Base Sepolia	0x00014A34	EIP-155 chain ID 84532	Base Sepolia testnet
Optimism	0x0000000A	EIP-155 chain ID 10 (0x0A)	OP Mainnet (Optimistic); 100-block BTC confirmation required
zkSync Era	0x00000144	EIP-155 chain ID 324 (0x144)	zkSync Era (ZK Rollup); 6-block BTC + L1 ZK proof finalization required
Polygon zkEVM	0x0000044D	EIP-155 chain ID 1101 (0x044D)	Polygon zkEVM (ZK Rollup); 6-block BTC + L1 proof required
Solana Mainnet	0x00001000	—	Solana Mainnet-Beta; 150 slot confirmation required
Solana Devnet	0x00001001	—	Solana Devnet; development/testing only
Solana Testnet	0x00001002	—	Solana Testnet; development/testing only
Cosmos Hub	0x00002000	IBC chain-id "cosmoshub-4"	Cosmos Hub Mainnet; IBC light client must be updated before finality
Osmosis	0x00002001	IBC chain-id "osmosis-1"	Osmosis DEX chain
Celestia Mainnet	0x00003000	—	Celestia data availability layer

Taproot + Ordinal Commitment Specification

Universal Bridge (URIB)

Deterministic Commitment, Satoshi-Level Anchoring, and Cross-Rail Truth Binding

NOTE — Chain ID Namespace

UB chain IDs

0x00000001

—

0x0000FFFF

are reserved for Bitcoin variants and EVM-compatible chains.

0x00001000

—

0x00001FFF

are reserved for Solana variants.

0x00002000

—

0x00002FFF

are reserved for Cosmos ecosystem chains.

0x00003000

—

0x00003FFF

are reserved for data availability layers. Values

0xFF000000

Taproot + Ordinal Commitment Specification

Universal Bridge (URIB)

Deterministic Commitment, Satoshi-Level Anchoring, and Cross-Rail Truth Binding

—

0xFFFFFFFF

are reserved for private/testnet use by bridge operators and

MUST NOT

appear in mainnet commitments.

Appendix C — Glossary Index

Alphabetical index of all defined terms with section references. Terms defined in Section 1.2 are marked (§1.2); terms introduced in subsequent sections are marked with their first-definition section.

Term	Defined In	Short Definition
Anchor UTXO	§1.2	Bitcoin P2TR output whose scriptPubKey encodes the UB commitment hash
Annex	§5.5	Optional final witness element prefixed 0x50; carries UBCommitRecord and payload
bind_hash	§1.2, §4.3	H_UBOrdBind output binding sat ordinal to payload; used as Taproot Merkle root
Chain of Custody	§6.4	Tamper-evident linked list of anchors via chain_link_hash field in payload_hash
Commitment Payload (C)	§1.2	Variable-length cross-chain state blob being anchored to Bitcoin
Control Block	§1.2, §5.3	Witness element for script-path spend: (leaf_ver parity) P merkle_path
Cross-Rail Anchor	§1.2, §6.1	Anchor UTXO whose payload commits to state on two or more non-Bitcoin rails
Dust Limit	§1.2, §4.5	Minimum P2TR output value: 546 sat (common) or 1 sat (rare+)
FIFO Transfer Rule	§3.2	Ordinals flow through transactions in First-In, First-Out order by input then output index
H_TapBranch	§1.1	tagged_hash with tag "TapBranch" (BIP-341)
H_TapLeaf	§1.1	tagged_hash with tag "TapLeaf" (BIP-341)
H_TapTweak	§1.1	tagged_hash with tag "TapTweak" (BIP-341)
H_UBCommit	§1.1	tagged_hash with tag "UB/Commit/v1"; used in keypath-only mode
H_UBOrdBind	§1.1	tagged_hash with tag "UB/OrdBind/v1"; used in sat-payload binding
Internal Key (P)	§1.2	32-byte x-only secp256k1 public key before tweak is applied

Taproot + Ordinal Commitment Specification

Universal Bridge (URIB)

Deterministic Commitment, Satoshi-Level Anchoring, and Cross-Rail Truth Binding

Term	Defined In	Short Definition
Leaf Version	§2.2	Single byte preceding script in TapLeaf serialization; 0xC0 for all UB leaves
Merkle Root (h)	§1.2	32-byte root hash of TapTree T; input to Taproot tweak alongside P
MultiRailPayload	§6.2	Structured payload format for cross-rail anchors with per-rail state proofs
NUMS Point	§1.2, §2.1	Nothing-Up-My-Sleeve secp256k1 point with no known discrete log; disables key-path spend
Ordinal	§1.2, §3.1	Globally unique 64-bit integer assigned to each satoshi in mining order, starting at 0
Output Key (Q)	§1.2	Tweaked x-only secp256k1 public key embedded in P2TR scriptPubKey; $Q = P + t \cdot G$ (even-Y normalized)
Parity Bit	§1.2, §2.5	Bit indicating odd-Y of Q before normalization; encoded in control block first byte
Rail	§1.2, §6.1	A distinct blockchain network participating in the Universal Bridge
Rarity Classes	§3.3	Six tiers of ordinal sat significance: common, uncommon, rare, epic, legendary, mythic
Recovery Script	§6.7	Emergency TapTree leaf allowing operator fund recovery after expiry + 1008 blocks
Reveal Transaction	§1.2, §5.3	Bitcoin transaction spending the commit UTXO, exposing UBCCommitRecord and payload in witness
Satoshi Binding	§1.2, §4.3	Cryptographic link between a specific ordinal sat and a commitment payload via H_UBOrdBind
Stability Window	§1.2, §3.5	Minimum 6-block confirmation depth on commit UTXO before reveal broadcast
TapBranch	§1.2, §2.3	Interior TapTree node; H_TapBranch of two children in lexicographic order
TapLeaf	§1.2, §2.2	TapTree leaf node; H_TapLeaf of (leaf_version compact_size(len) script)
TapTree (T)	§1.2, §2.4	Binary Merkle tree of Tapscript spending conditions

Taproot + Ordinal Commitment Specification

Universal Bridge (URIB)

Deterministic Commitment, Satoshi-Level Anchoring, and Cross-Rail Truth Binding

Term	Defined In	Short Definition
tagged_hash	§1.1	SHA256(SHA256(tag) SHA256(tag) msg); domain-separated hash per BIP-340
UBCommitRecord	§4.1	Fixed 167-byte binary structure containing all UB commitment metadata and signature
UB_COMMIT_SCRIPT	§5.4	Canonical Tapscript template for script-path UB commitment leaves (leaf_version 0xC0)

AMENDMENT NOTE — VERSION 1.1 (September 20, 2026)

Taproot + Ordinal Commitment Specification — Universal Bridge (UB-SPEC-BTC-001)

This version corrects three technical errors identified in a full audit pass of Version 1.0.0-draft. All corrections are listed here; the Version 1.0.0-draft text (preserved in the Universal Bridge Archive, DOI 10.5281/zenodo.22852641) remains accessible as the historical record.

1. Merkle proof path length for depth-2 leaves (Section 5.3 and Appendix A)

Version 1.0.0-draft states that depth-2 leaves of the 3-leaf TapTree require one sibling hash (32 bytes). Under BIP-341, a leaf at depth d requires exactly d sibling hashes ($32 \times d$ bytes): depth-2 leaves require two sibling hashes (64 bytes — the depth-2 sibling concatenated with the depth-1 sibling), yielding a control block of $1 + 32 + 64 = 97$ bytes, not 65 bytes. As previously written, consensus validation would compute the wrong leaf depth from the control block length and reject the spend. Sections 5.3 and Appendix A are corrected in this version.

2. TapTreeMerkleRoot pseudocode ordering (Section 2.4)

Version 1.0.0-draft prepends each computed parent node at the front of the node list (`nodes.insert(0, parent)`), which breaks the depth-descending processing order when multiple sibling pairs exist at the same depth and trips the sibling-depth assertion. The corrected algorithm appends the parent (`nodes.append(parent)`), which preserves the depth-descending invariant: remaining deeper nodes are consumed before the shallower parents. Section 2.4 is corrected in this version.

3. first_sat example arithmetic (Section 3.1)

The example read `first_sat(210000) = 1,050,000,000,000,000,000`; the correct value is `first_sat(210000) = 1,050,000,000,000,000` ($210,000 \text{ blocks} \times 50 \text{ BTC} \times 10^8 \text{ satoshis/BTC} = 1.05 \times 10^{15} \text{ satoshis}$). The three trailing zeroes were erroneous; the supply figure in Table 3.3 was already correct. Section 3.1 is corrected in this version.

Implementer advisory (standardness; requirements unchanged)

Carriers carrying exactly 1 satoshi (DUST_LIMIT_RARE) are below Bitcoin Core's standard P2TR mempool relay limit (330 satoshis). Such transactions will not relay through default policy and require direct-to-miner or custom relay submission channels. This advisory is informational; the specification's normative requirements are unchanged.

Publication note: per-page footer armor (record DOI + ORCID + prototype disclaimer) is applied to all pages in this version. Fabrication screen: zero unbacked benchmarks; all figures are protocol constants or deterministic test vectors. DOI: 10.5281/zenodo.22854062 — Leon Calvin Long, II (ORCID 0009-0002-1140-9568).

This software is a prototype and is provided for educational and research purposes only. It is not intended for production use, commercial deployment, or safety-critical environments. All systems are experimental and may contain defects on them.